Your Iceberg Table Is Already a Pinot Table

A Netflix format, a LinkedIn engine, and why they fit



Gonzalo Ortiz · Apache Pinot PMC · StarTree

Quick show of hands



Written a Parquet file?



Know what is in its footer?



Used Iceberg in production?



Heard of Apache Pinot?

CHAPTER 1

Iceberg

A table format, built bottom-up



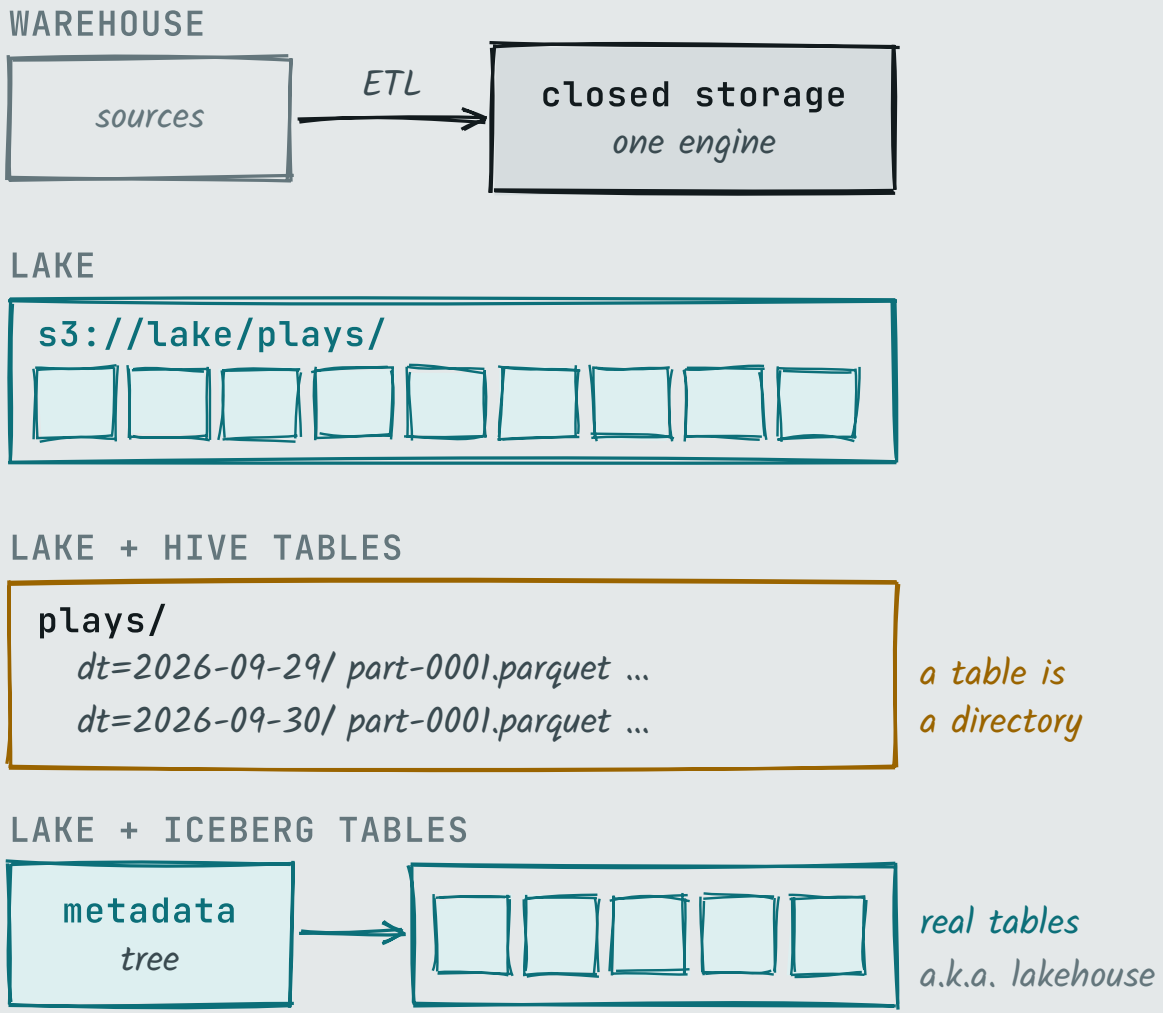
youtube.com

RECOMMENDED WATCH

Apache Iceberg: What It Is and Why Everyone's Talking About It

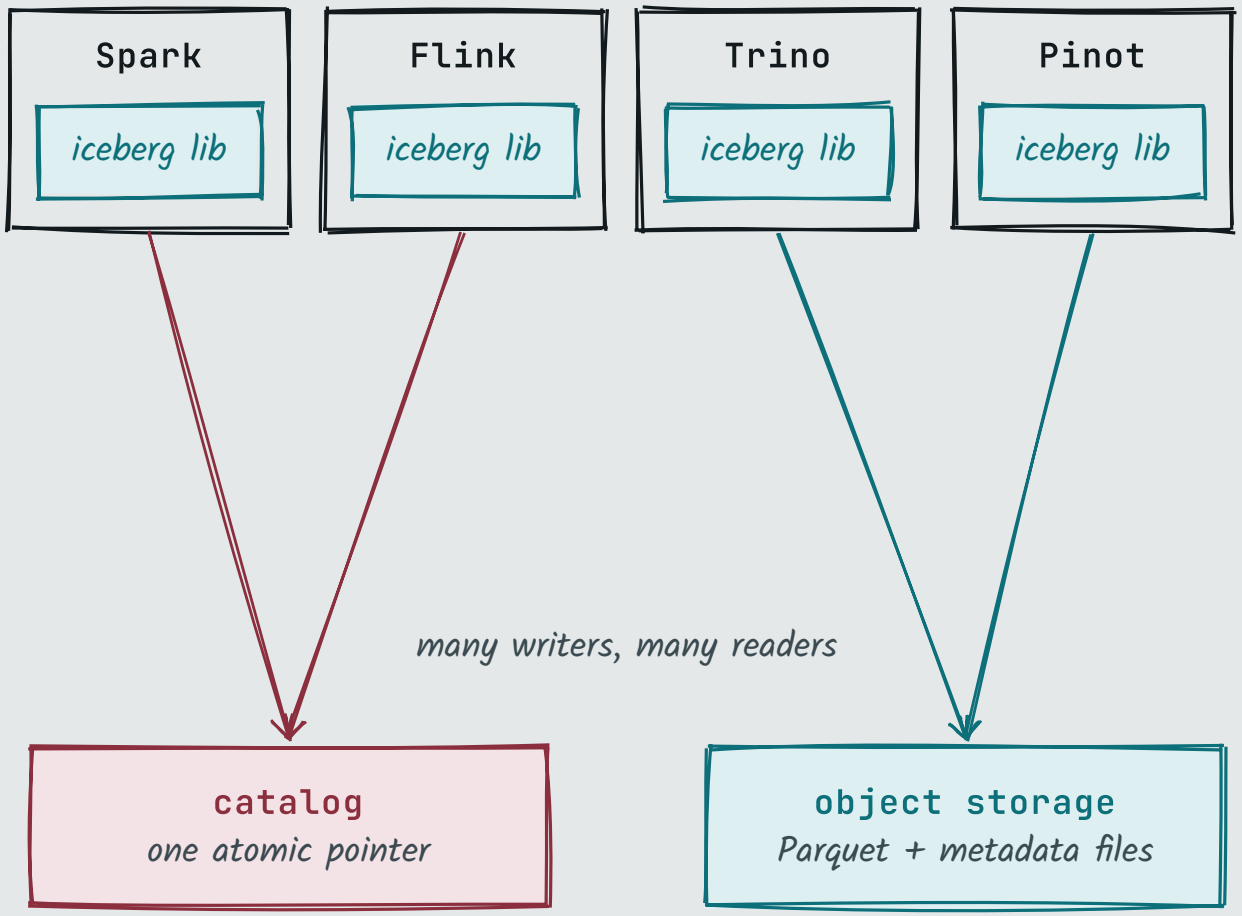
Confluent Developer · YouTube

Warehouse, lake, lakehouse



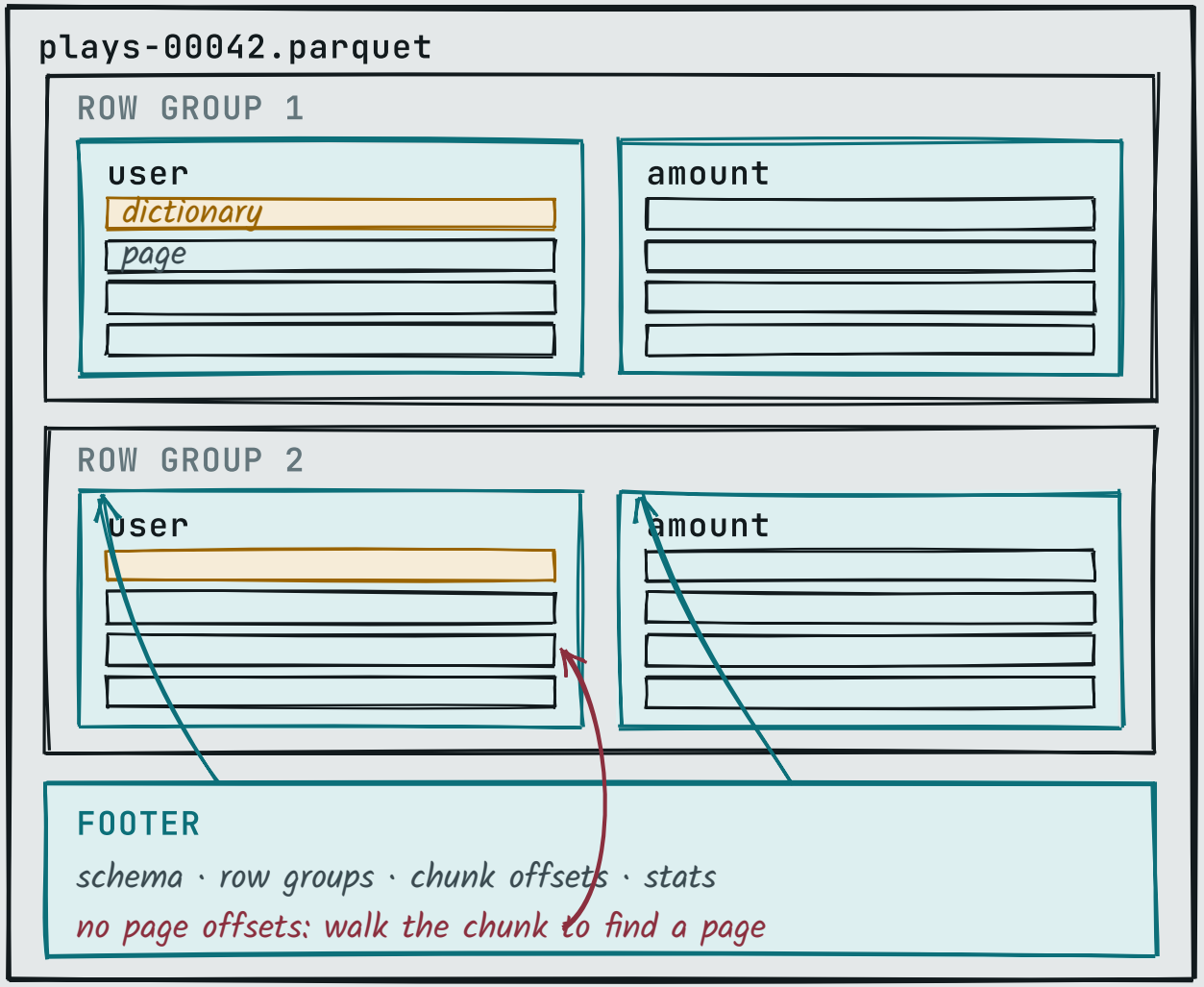
A format, not an engine

- No Iceberg server
- No coordinator
- A spec plus libraries
- Every engine embeds them
- Many writers, many readers



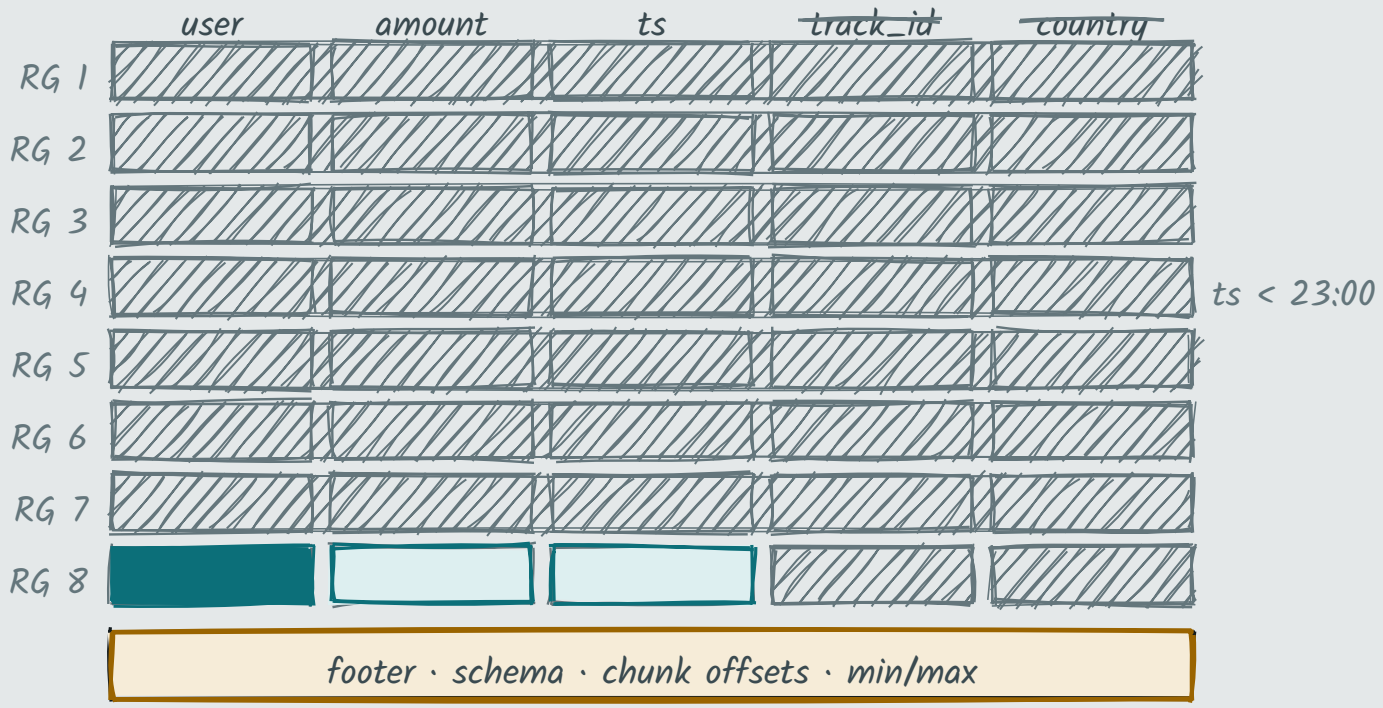
Parquet 101

- **Row group:** horizontal slice
- **Column chunk:** one column, contiguous
- **Page:** smallest readable unit
- **Footer:** offsets, stats, schema
- No page offsets in the footer



Reading one Parquet file

```
SELECT SUM(amount) FROM db.plays
WHERE user = 'bob' AND ts >= '2026-09-30 23:00'
```



BYTES READ

~20 MB

of a 512 MB file

Read ts and amount, sum the matching rows

user DICTIONARY · RG 8

'alice' → 11 · **'bob' → 12** · 'carol' → 13

SUM(amount) = 3,140

Building a table, bottom-up

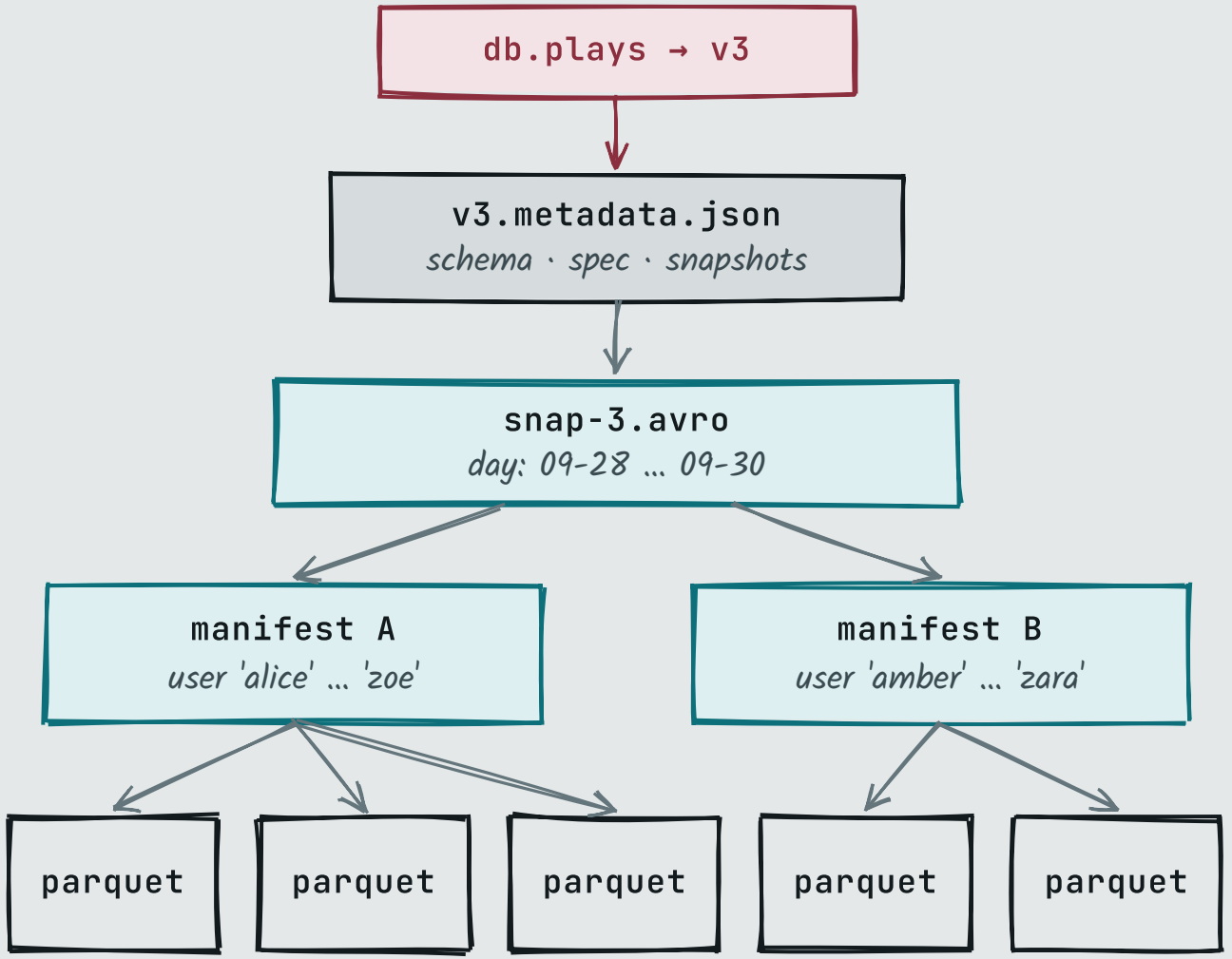
CATALOG

METADATA

MANIFEST LIST

MANIFESTS

DATA FILES



the only mutable thing

schemas · snapshots

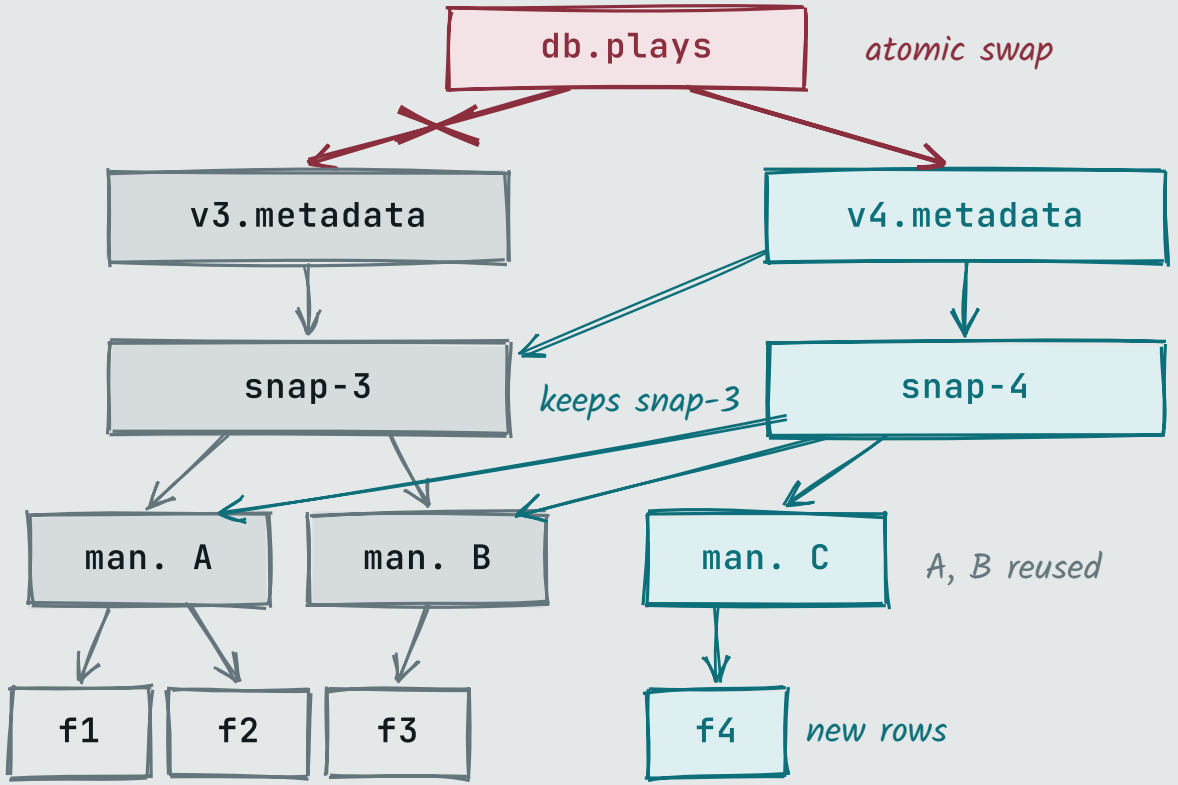
partition ranges

per-file column min/max

immutable Parquet

A commit

1. Write new data files
2. Write new manifests
3. Write new metadata.json
4. Swap the catalog pointer
5. Lost the race? Retry

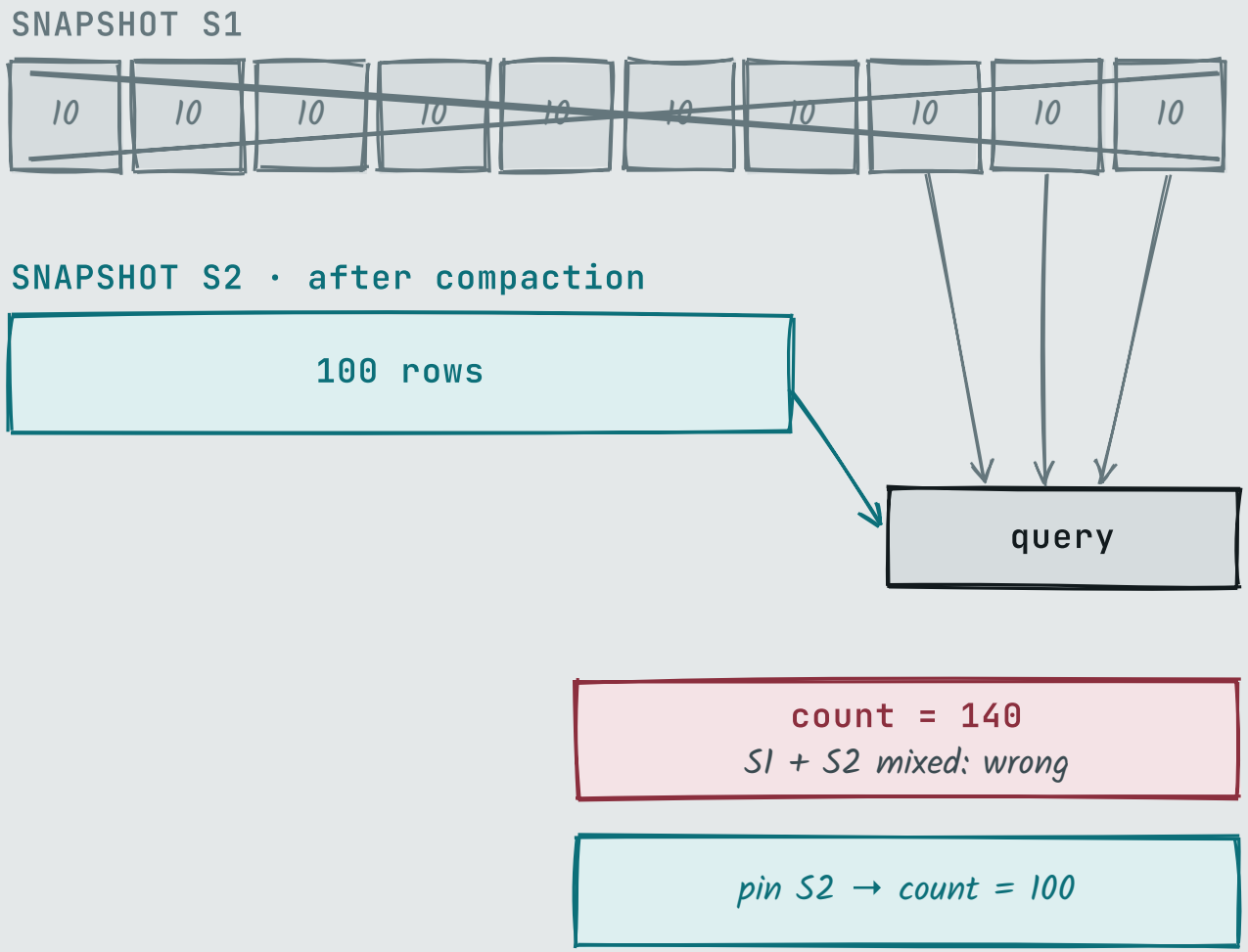


Someone swapped first? The swap fails.

Re-read the new tree, re-apply the change **if possible**, try again.

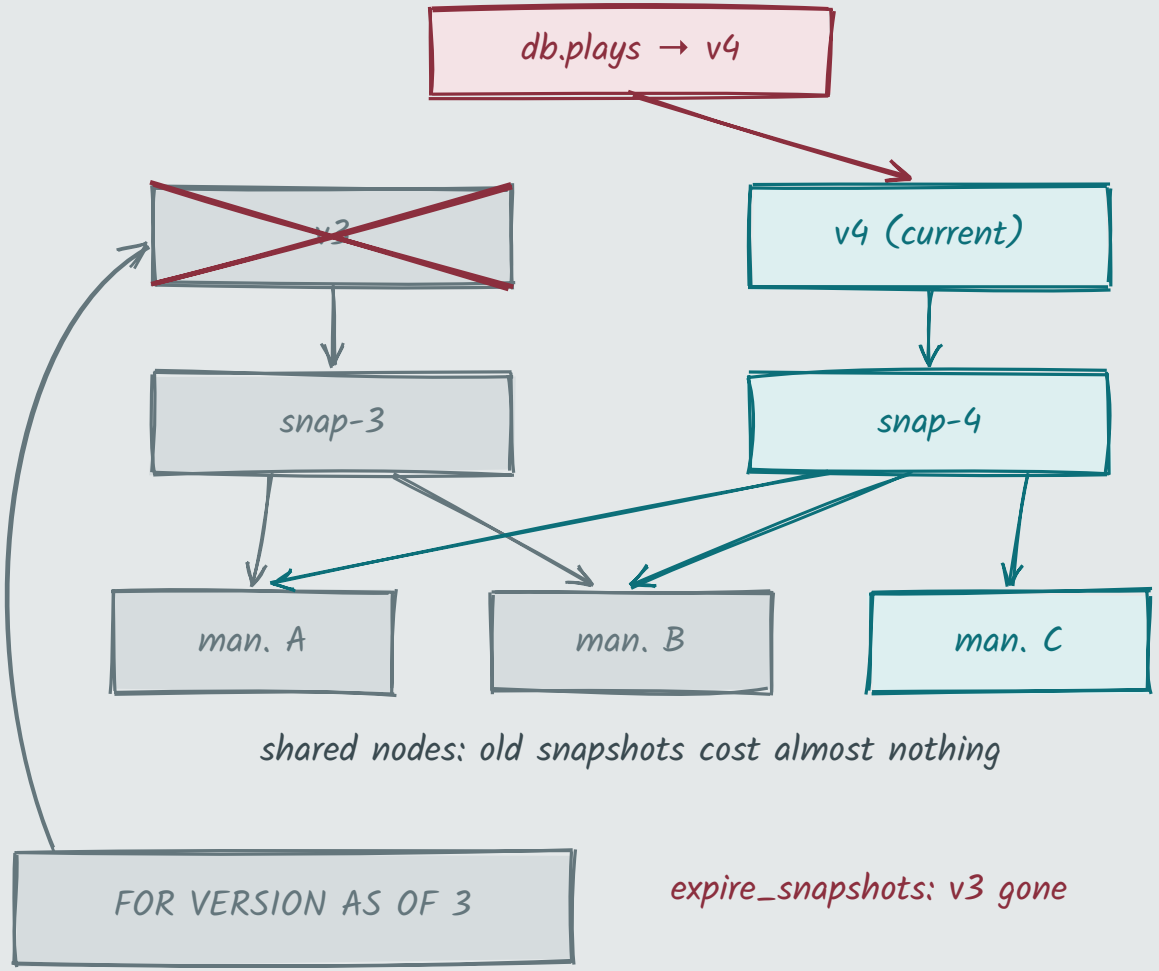
Never mix two snapshots

- Compaction: 10 files become 1
- Mix before and after?
- Rows twice, or not at all
- Every read pins one snapshot



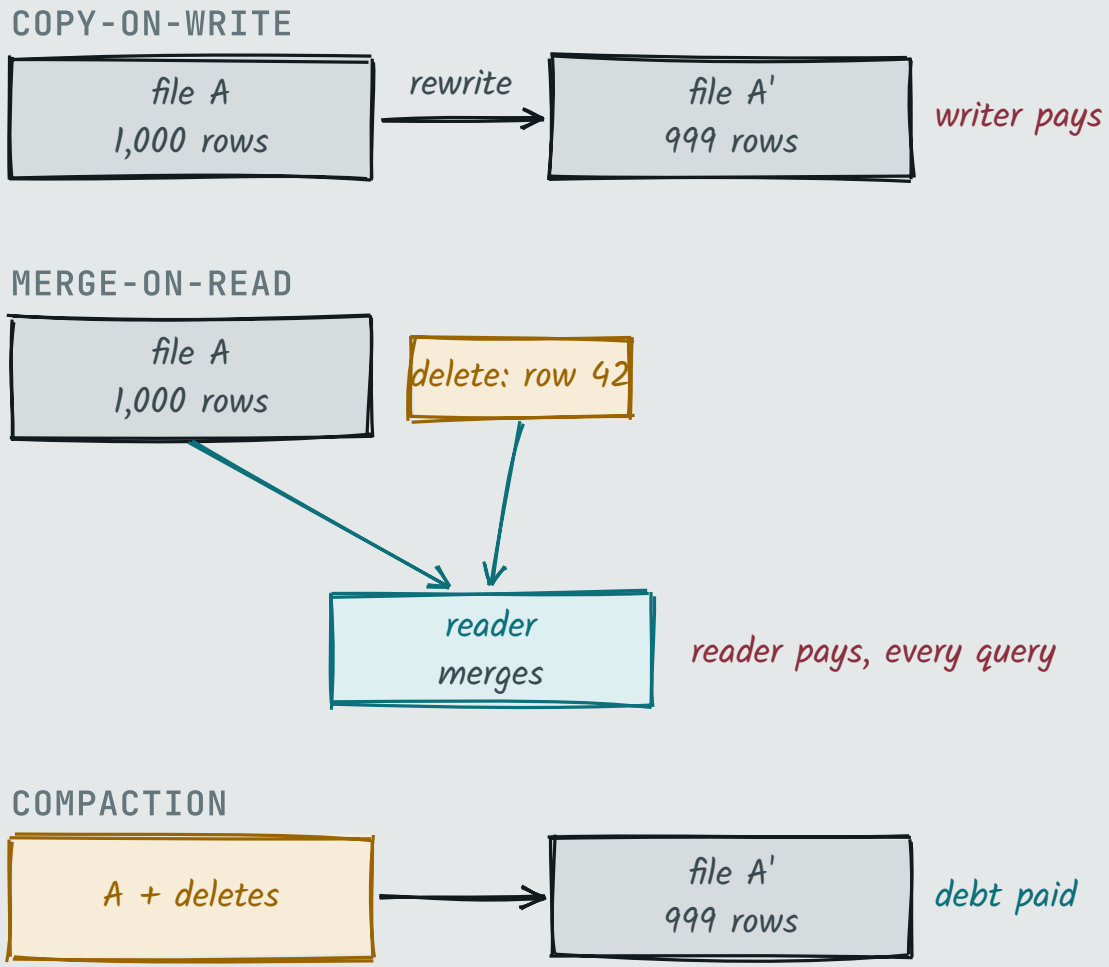
Time travel

- Old trees stay reachable
- `FOR VERSION AS OF`
- Branches and tags
- Until snapshots expire



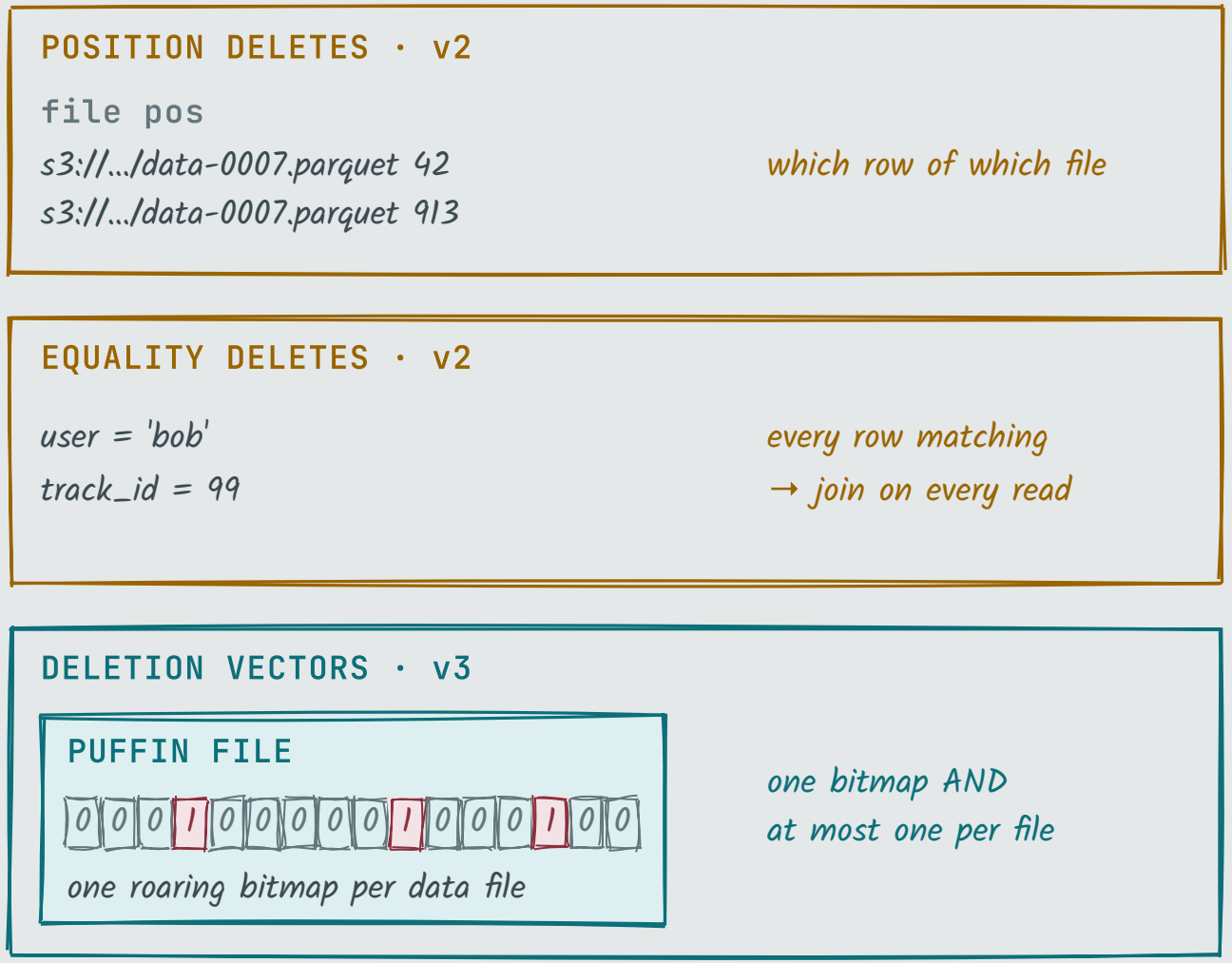
Deletes: who pays?

- **Copy-on-write:** writer rewrites files
- **Merge-on-read:** writer adds a delete file
- Reader merges at scan time
- Compaction folds them back



Merge-on-read, three ways

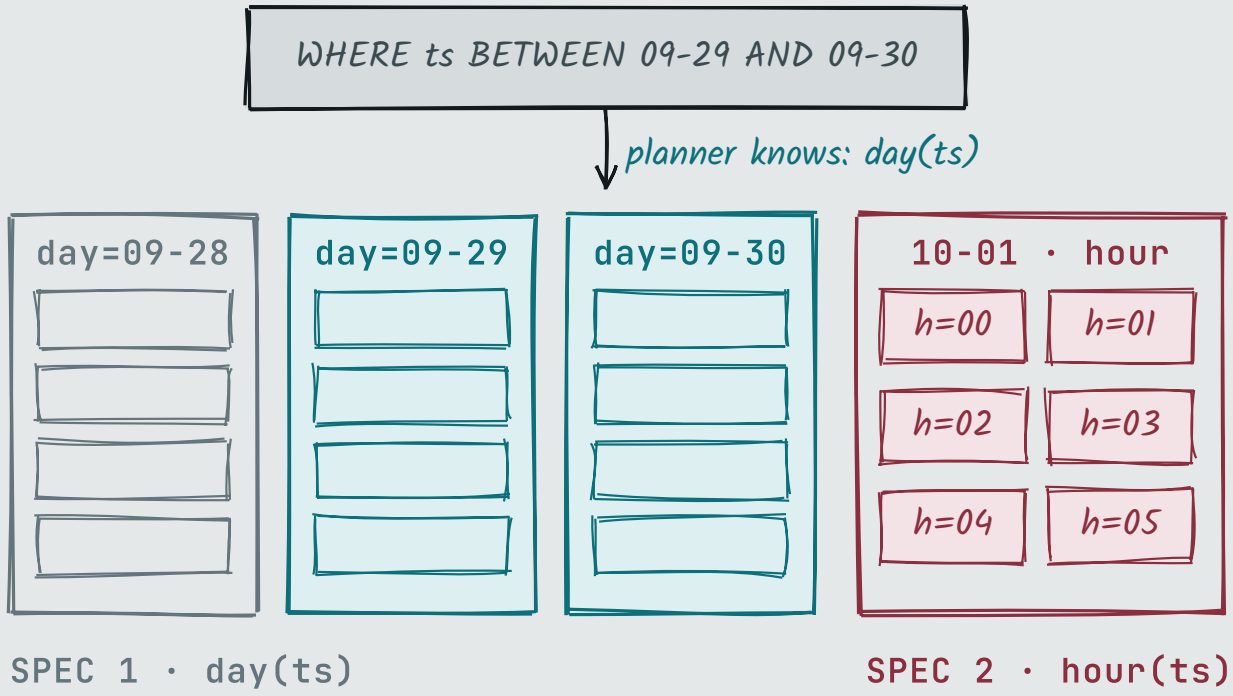
- **Position deletes** `v2` : file + row
- **Equality deletes** `v2` : "where id = 42"
- **Deletion vectors** `v3` : one bitmap per file



Puffin: a blob file format from the Iceberg spec. It stores DVs and statistics.

Hidden partitioning

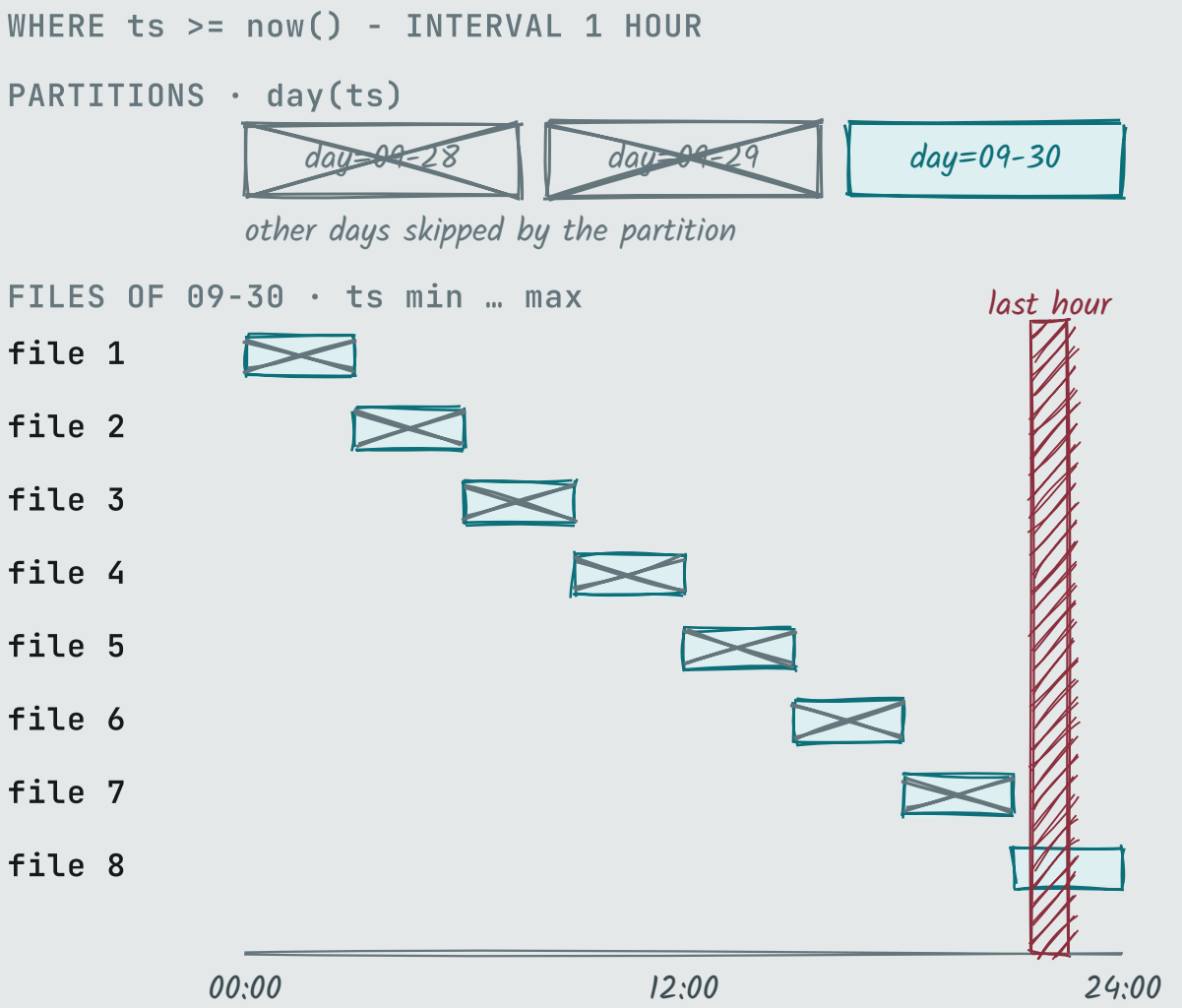
- Partition by a transform
- `day(ts)`, `bucket(16, user)`
- Users filter on `ts`
- Change the spec, keep old files



old files keep spec 1, nothing is rewritten

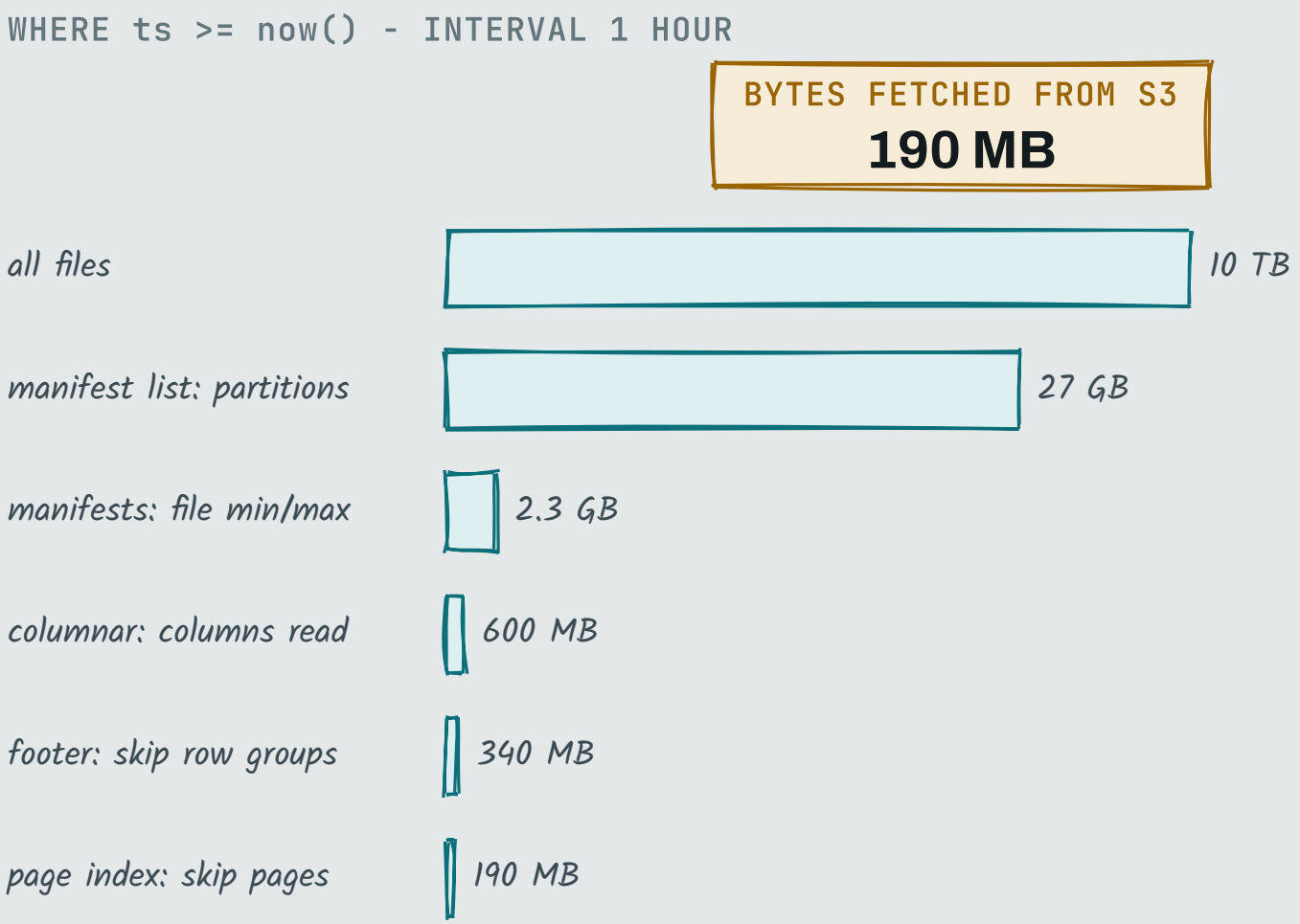
Pruning: ideal case

- Partition: assume `day(ts)` and `bucket(16, user)`
- Inside the day: files in time order
- Query: the last hour
- One file overlaps: skip the rest



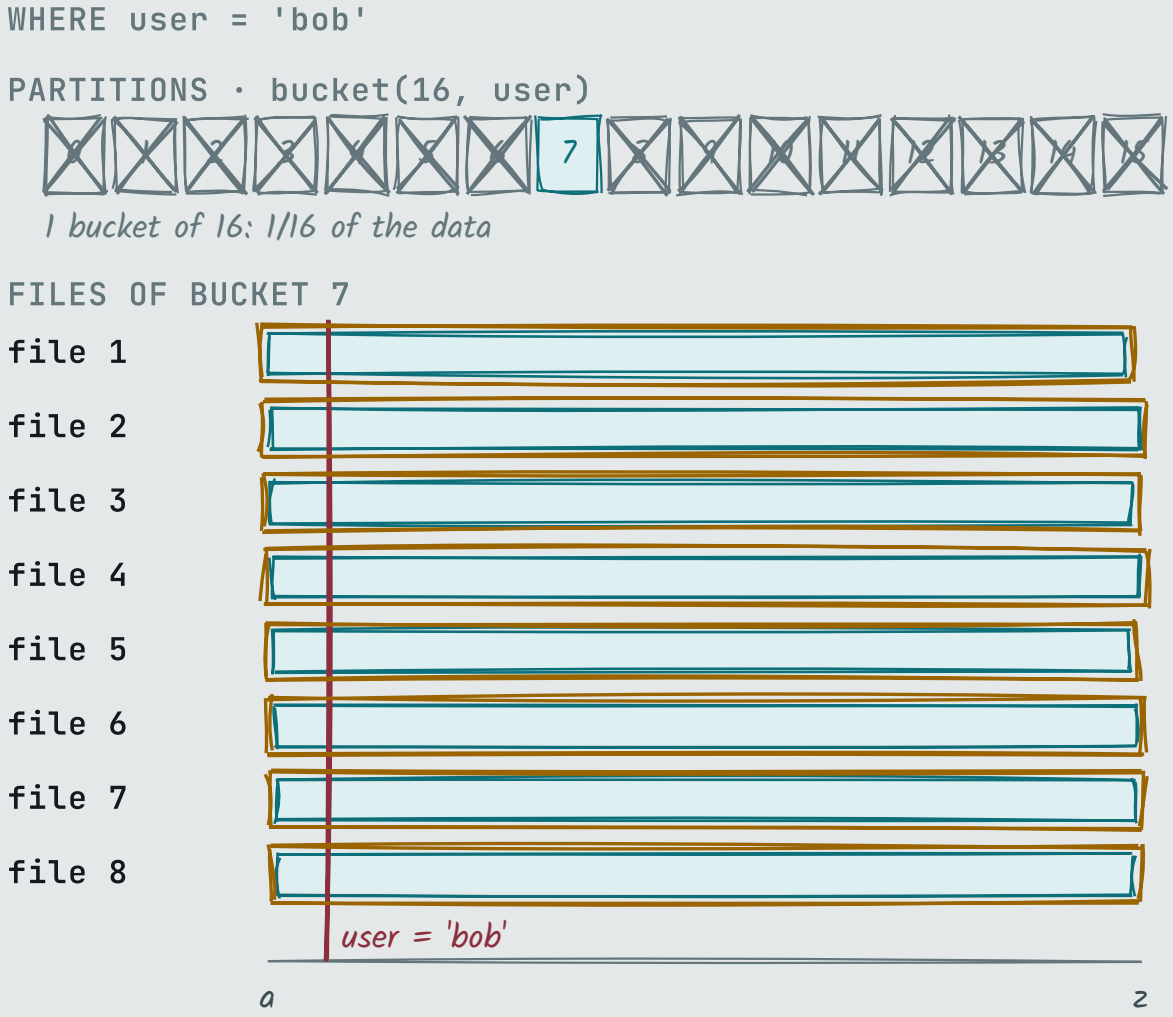
Pruning: ideal case

- Manifest list: partitions
- Manifests: file min/max
- Columnar: needed columns only
- Footer: skip row groups
- Page index: skip pages



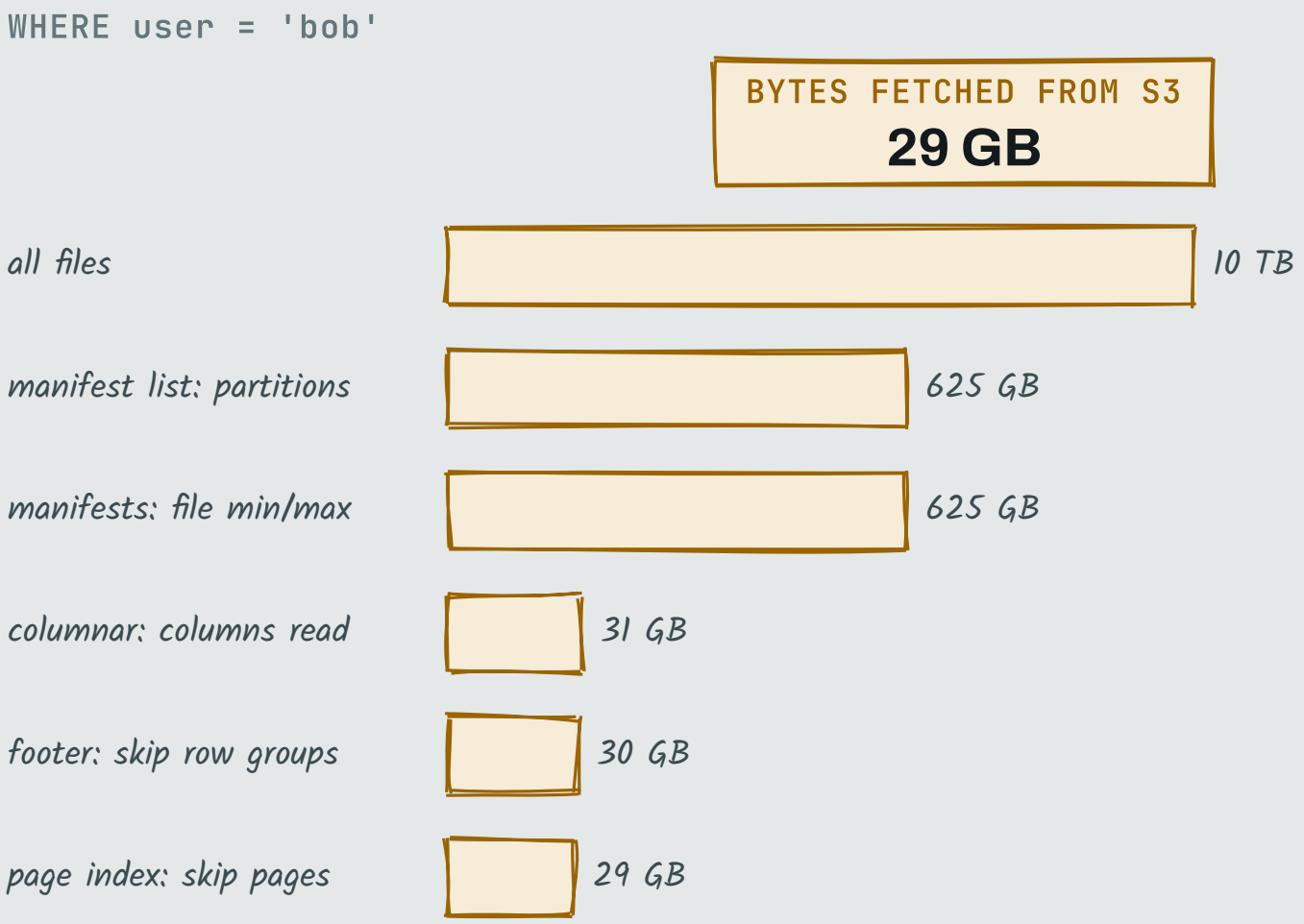
Pruning: hard case

- Partition: 1 bucket of 16
- Inside the bucket: `user` spread out
- Every range contains it
- Min/max skips nothing

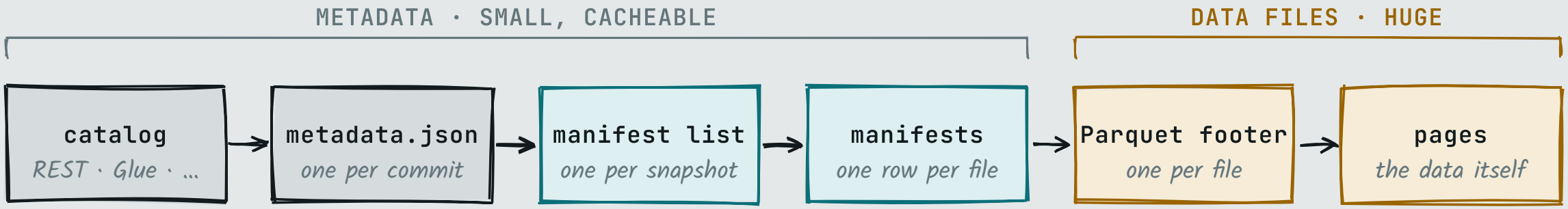


Pruning: hard case

- Filter on `user` only
- Bucket: 1/16 of the data
- File min/max: no help
- Columnar still helps
- ~29 GB for a few rows



How engines read Iceberg



ENGINE	LOCAL COPY	REMOTE SCAN + CACHE
Trino / Athena / Spark	—	✓
DuckDB	✓	✓
Dremio	✓ Reflections	✓ C3 cache
Snowflake / BigQuery	✓	✓
StarRocks / Doris / ClickHouse / Databricks	✓	✓ data / disk cache

Official documentation of each engine, checked 2026-10-01.

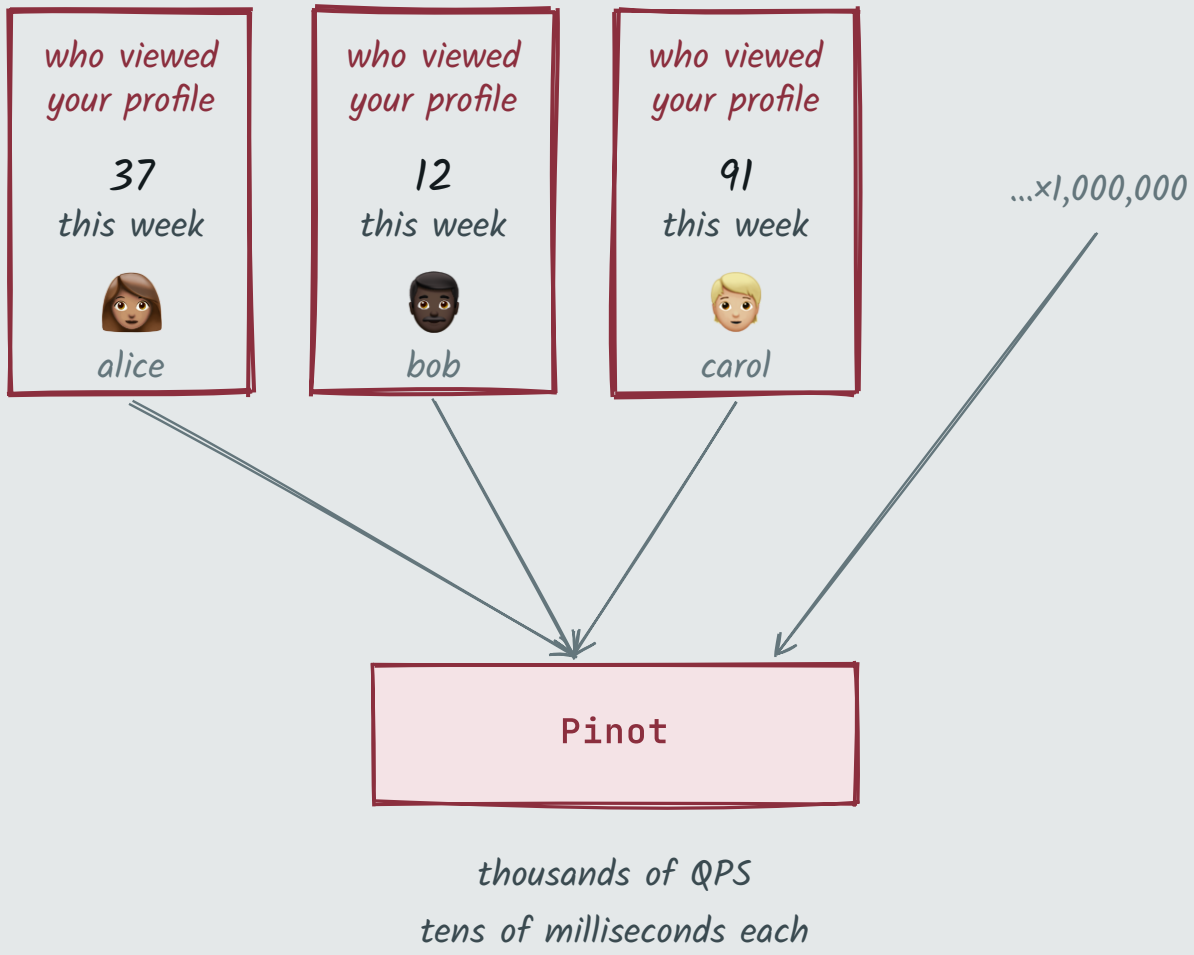
CHAPTER 2

Pinot

Analytics as a product feature

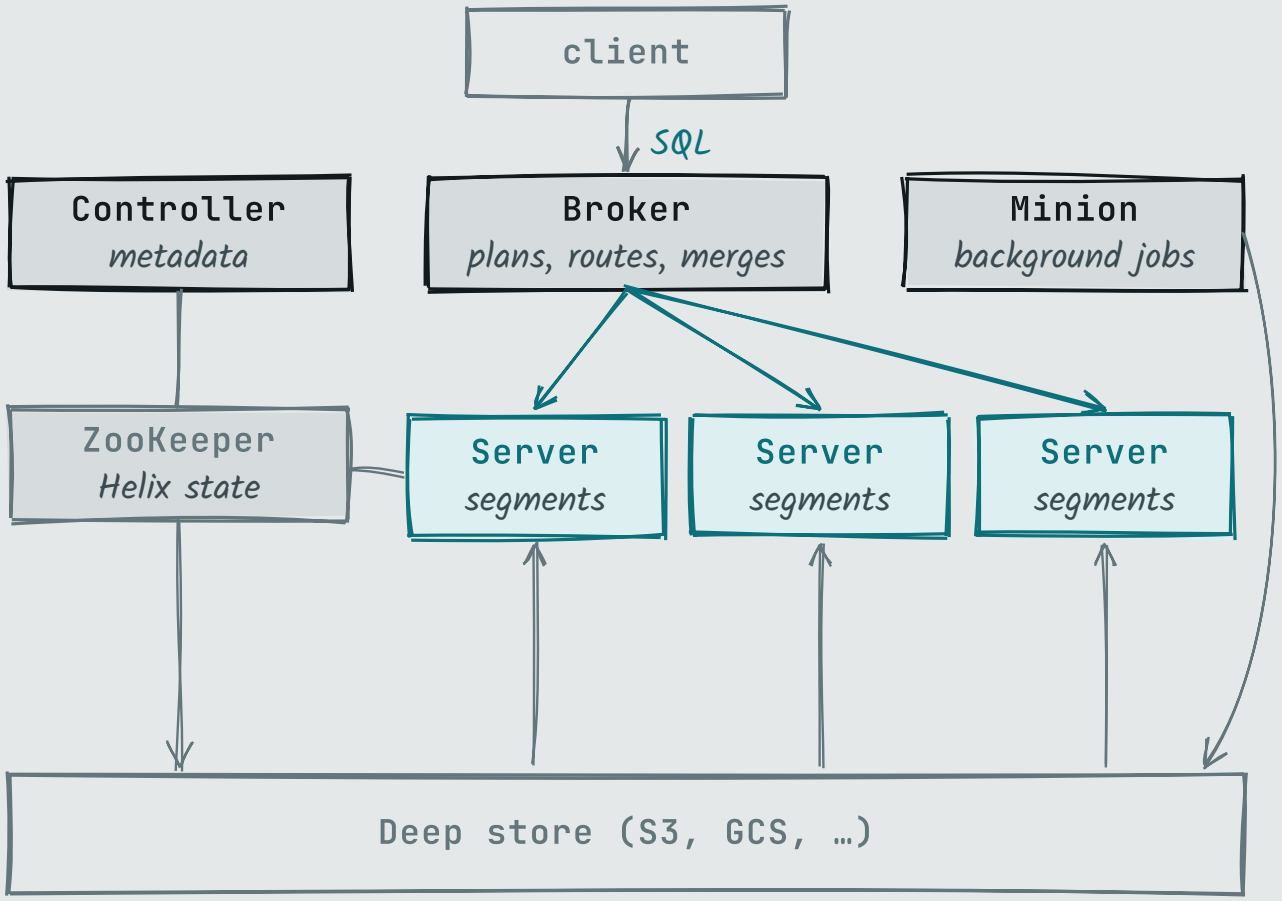


Why Pinot exists



The moving parts

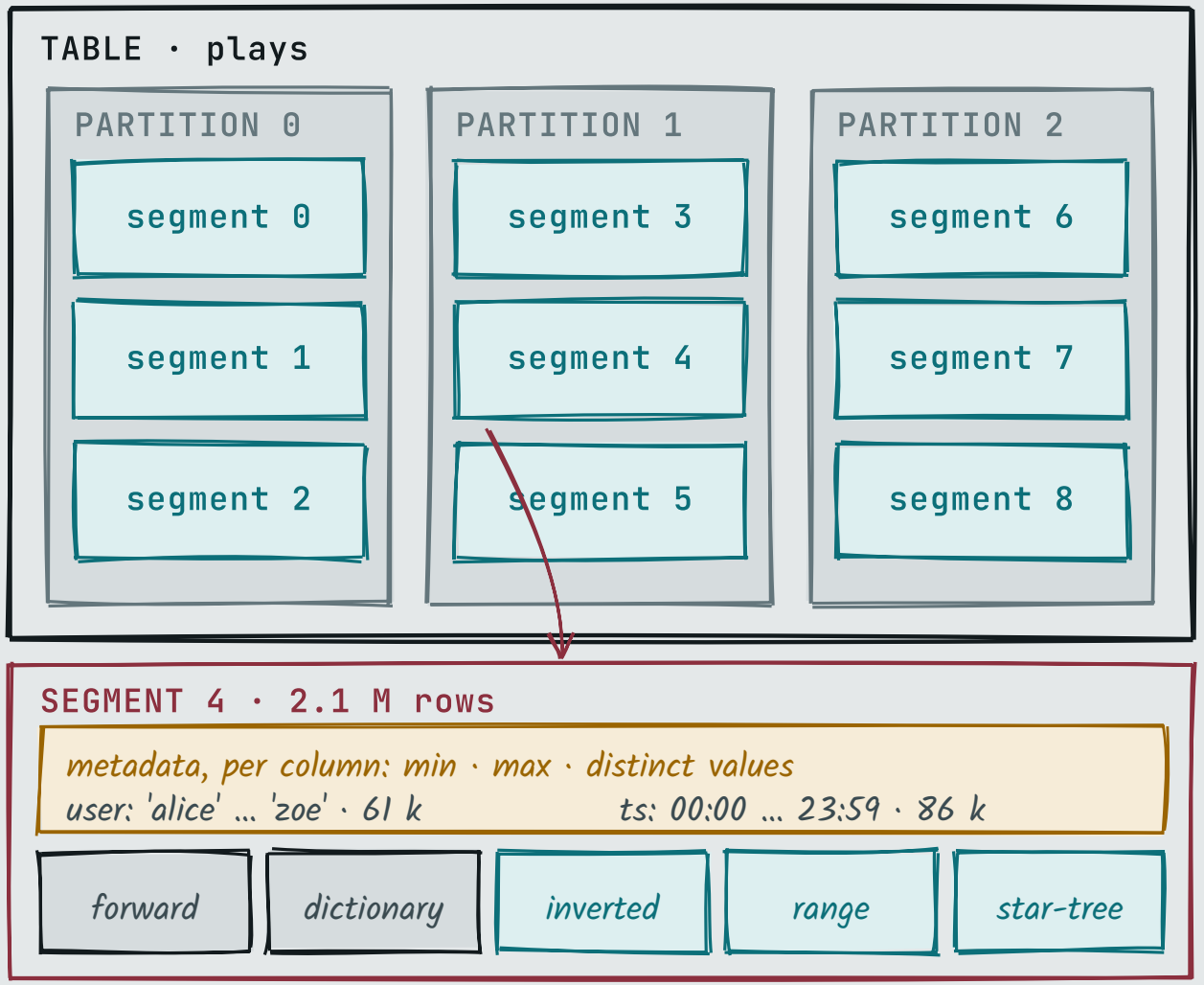
- **Controller:** metadata, assignment
- **Broker:** plans, routes, merges
- **Server:** stores, executes
- **Minion:** background jobs
- ZooKeeper + deep store



segments: immutable files in the deep store, loaded by servers

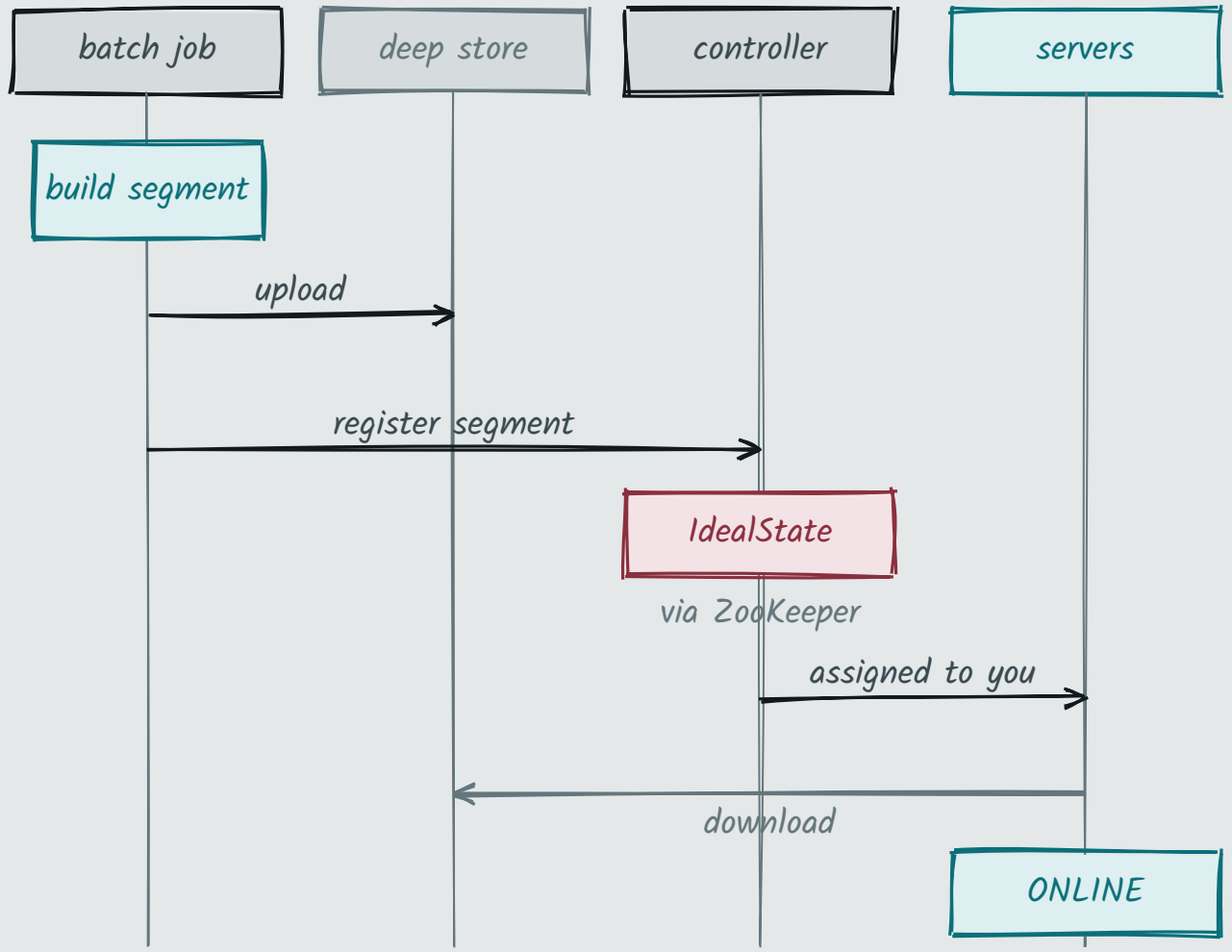
Tables all the way down

- Table
- Partitions
- Segments: immutable, columnar
- Metadata + indexes per column



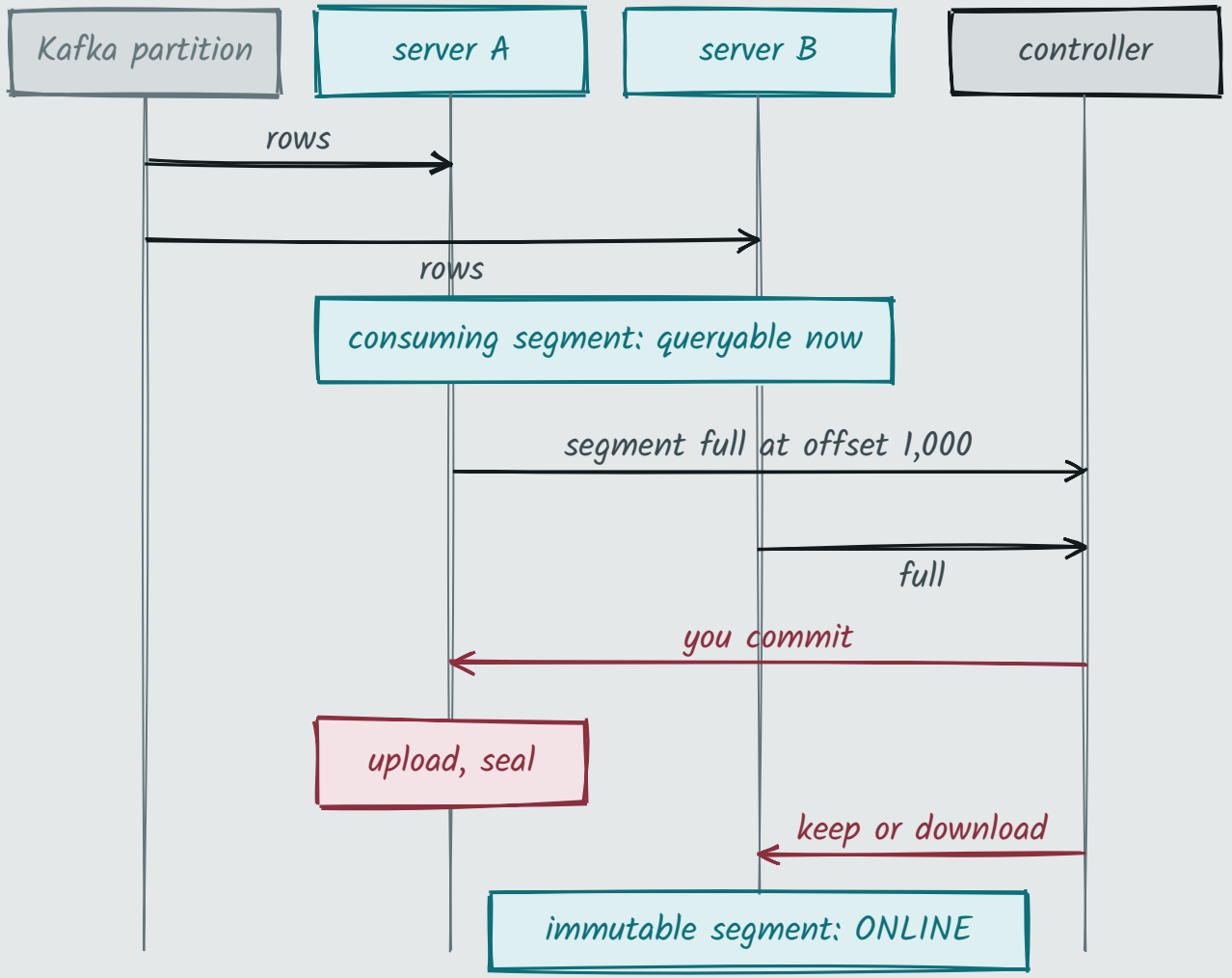
Offline ingestion

1. Job builds a segment
2. Upload to deep store
3. Controller registers it
4. Servers download and serve



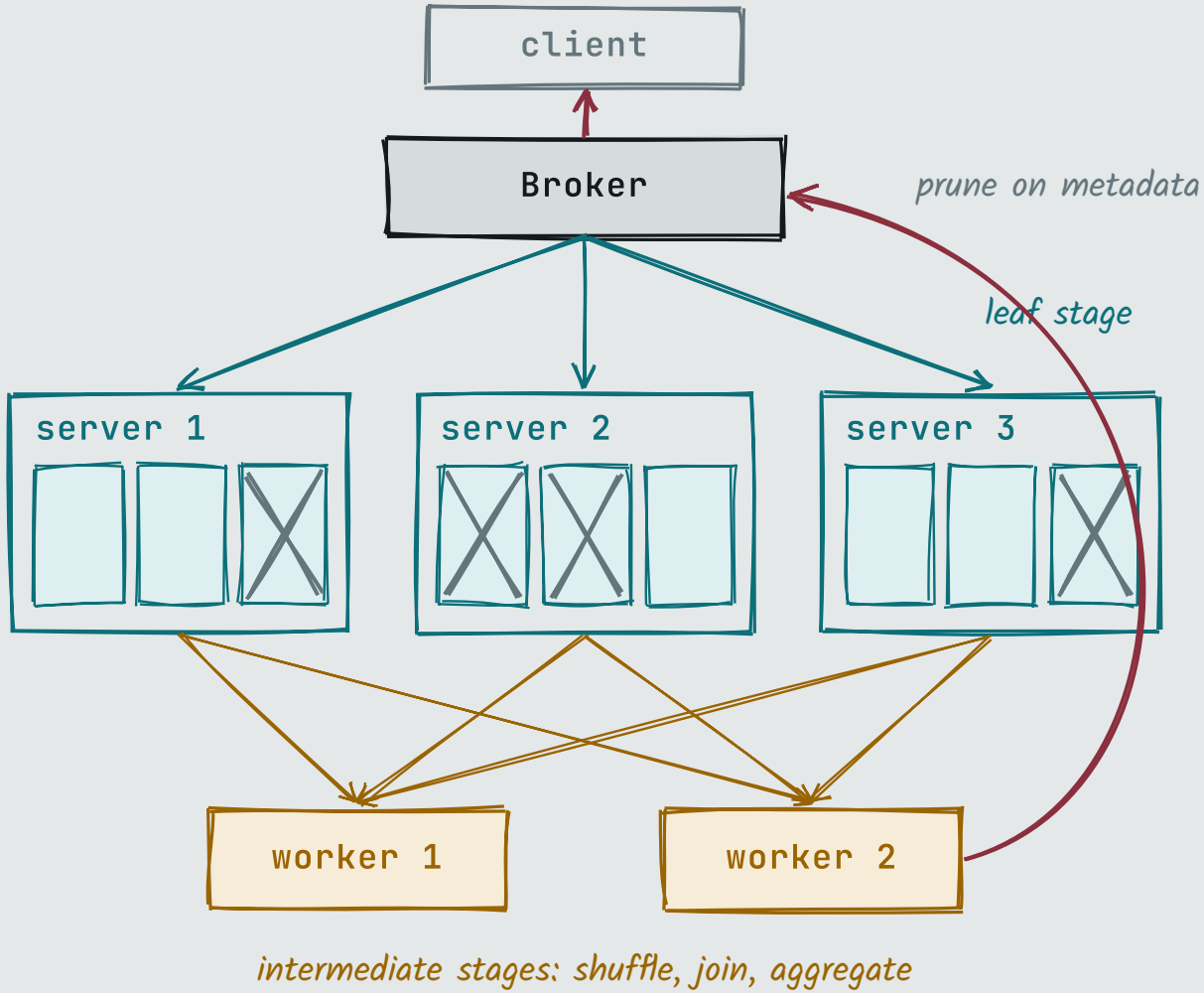
Realtime ingestion

1. Servers consume a Kafka partition
2. Rows are queryable at once
3. Segment fills up
4. One replica commits it
5. It becomes immutable



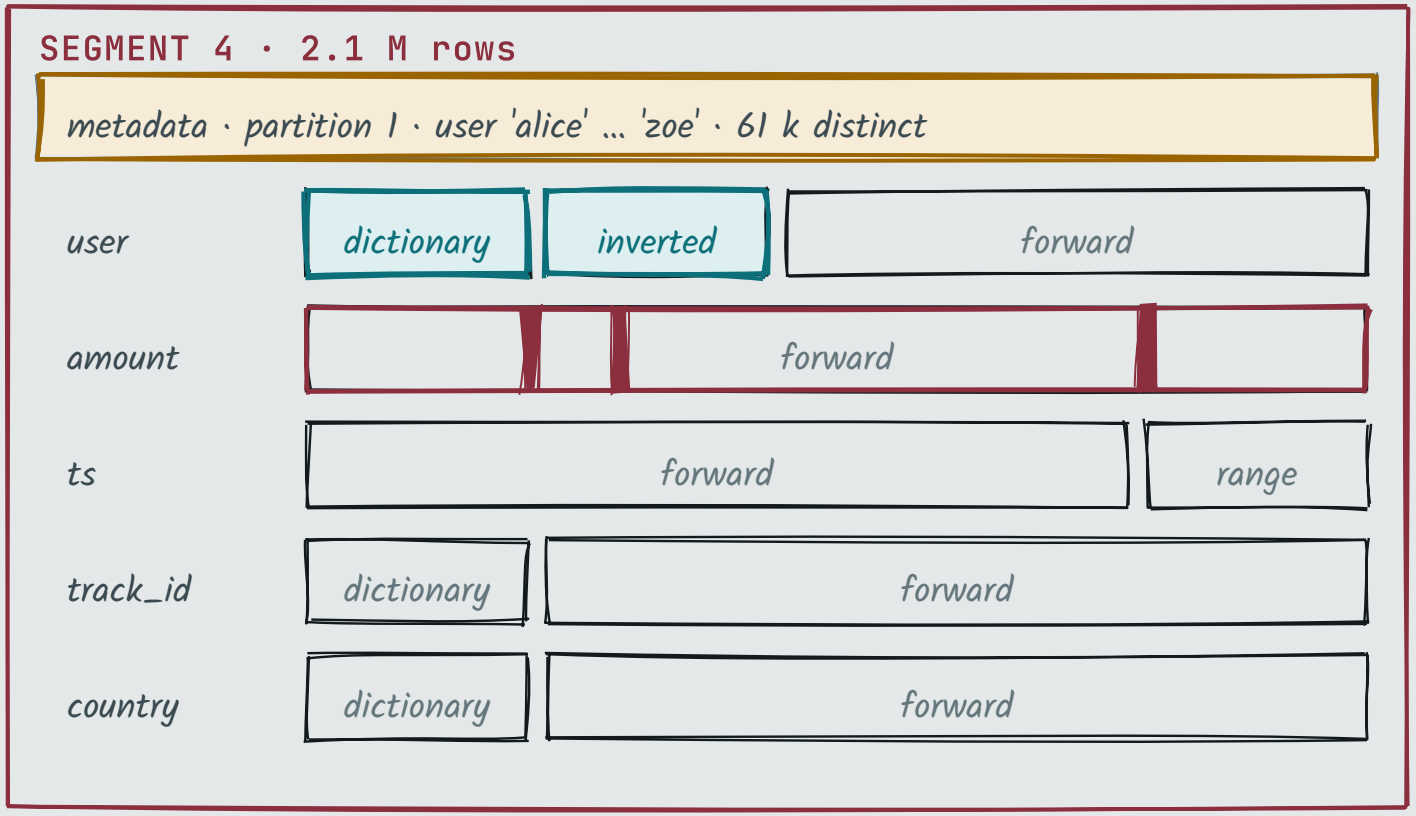
A SQL query, end to end

1. Broker prunes segments
2. Leaf stage: servers scan with indexes
3. Intermediate stages: shuffle, join, aggregate
4. Broker returns the result



Reading one Pinot segment

```
SELECT SUM(amount) FROM plays
WHERE user = 'bob'
```



ROWS SCANNED

3
of 2.1 M rows

Forward index: read 3 rows of amount

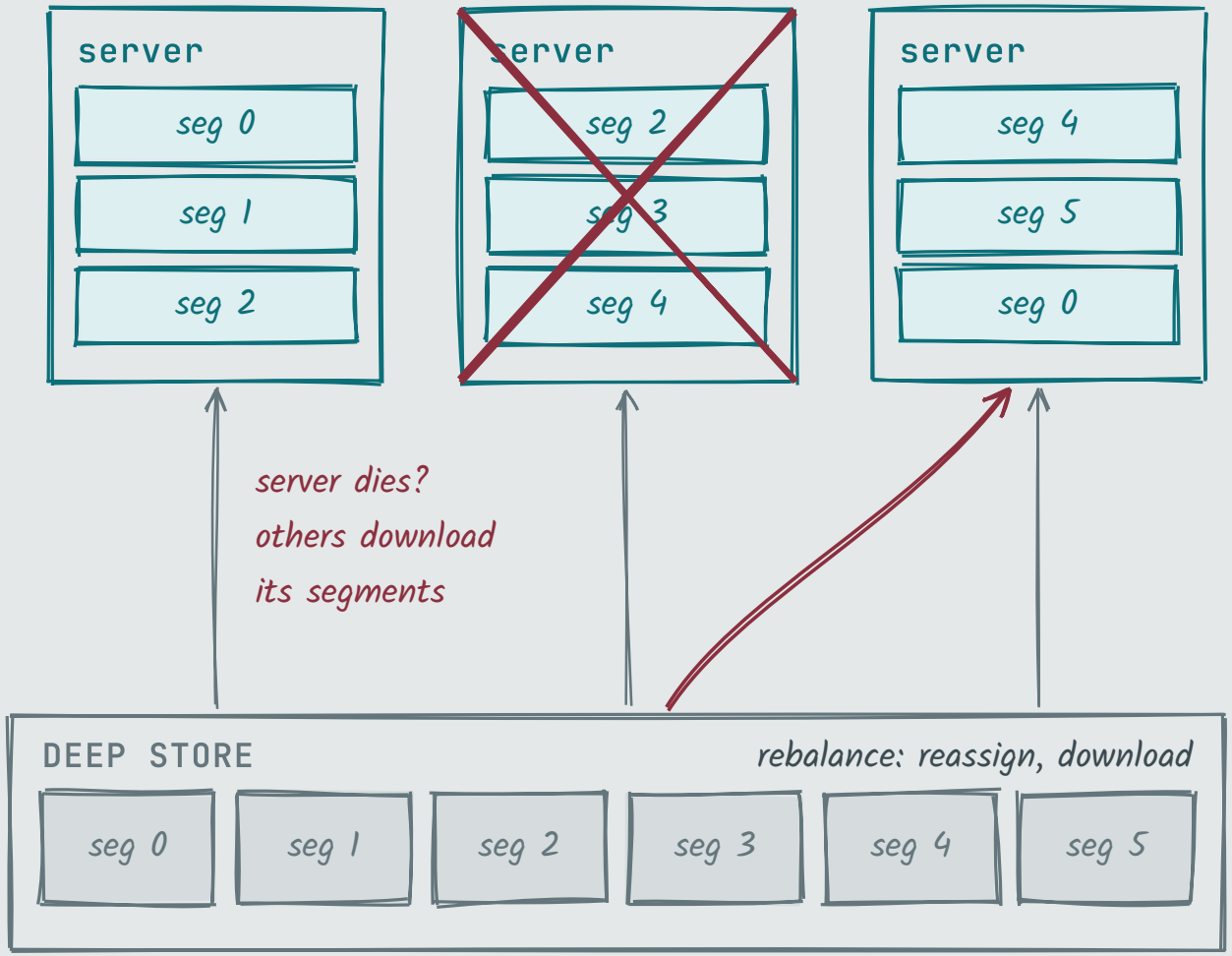
user DICTIONARY
'alice' → 11 · **'bob' → 12** · 'carol' → 13

user INVERTED · id 12
docs 20,412 · 20,981 · 61,003

SUM(amount) = 3,140

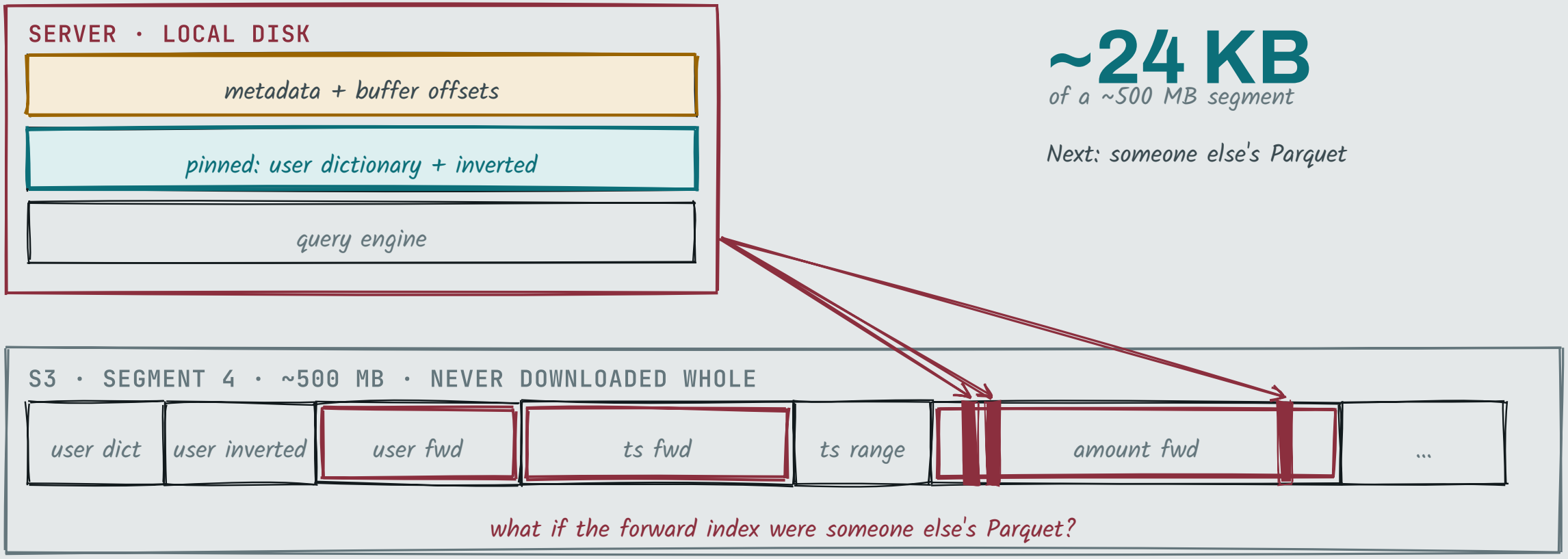
Segments live in the deep store

- Every segment is in the deep store
- Servers keep a local copy
- Server dies? Download again
- Rebalance: move assignments



Tiered storage

```
SELECT SUM(amount) FROM plays
WHERE user = 'bob'
```



BYTES FROM S3

~24 KB

of a ~500 MB segment

Next: someone else's Parquet

CHAPTER 3

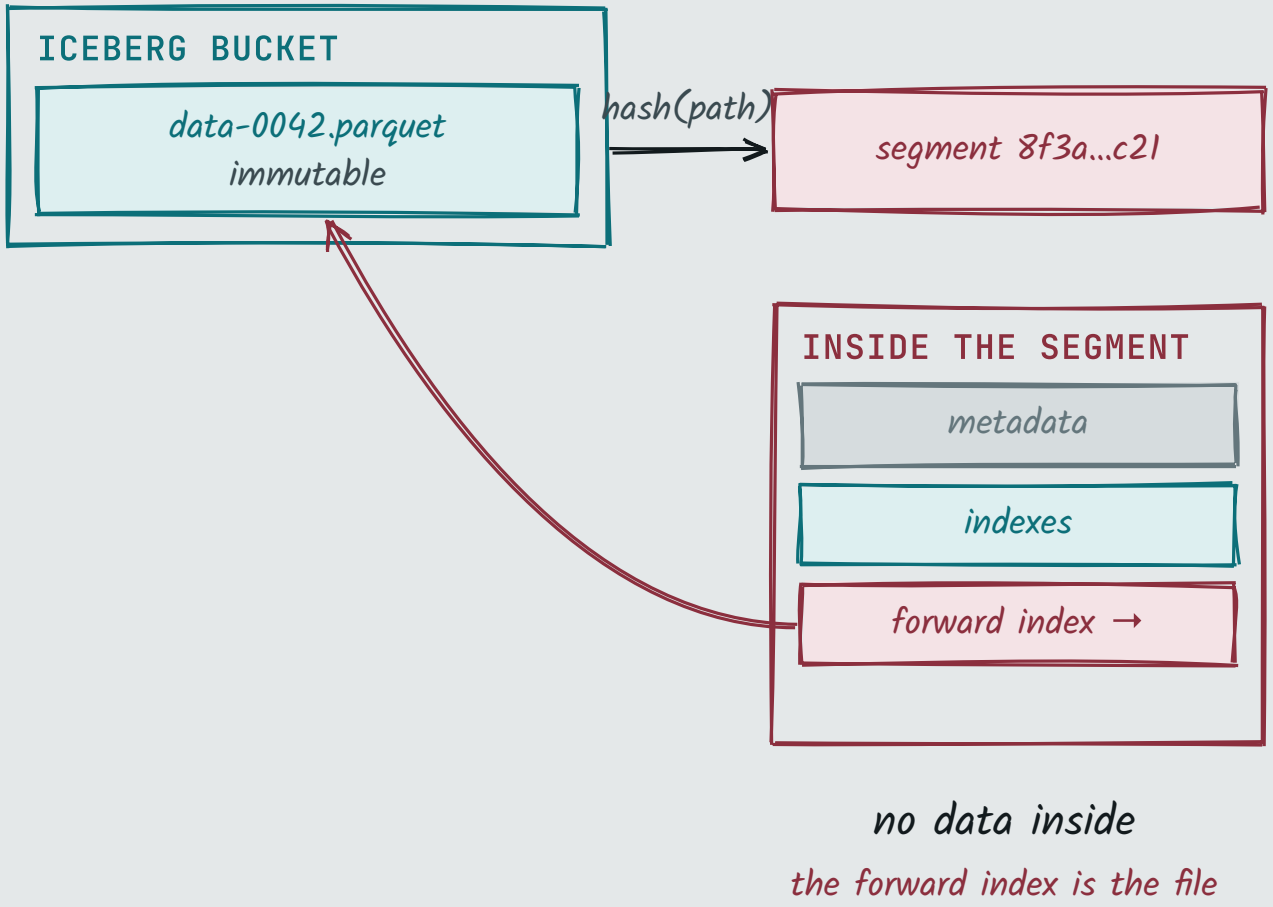
Iceberg ♥ Pinot

Same shape, so no copy



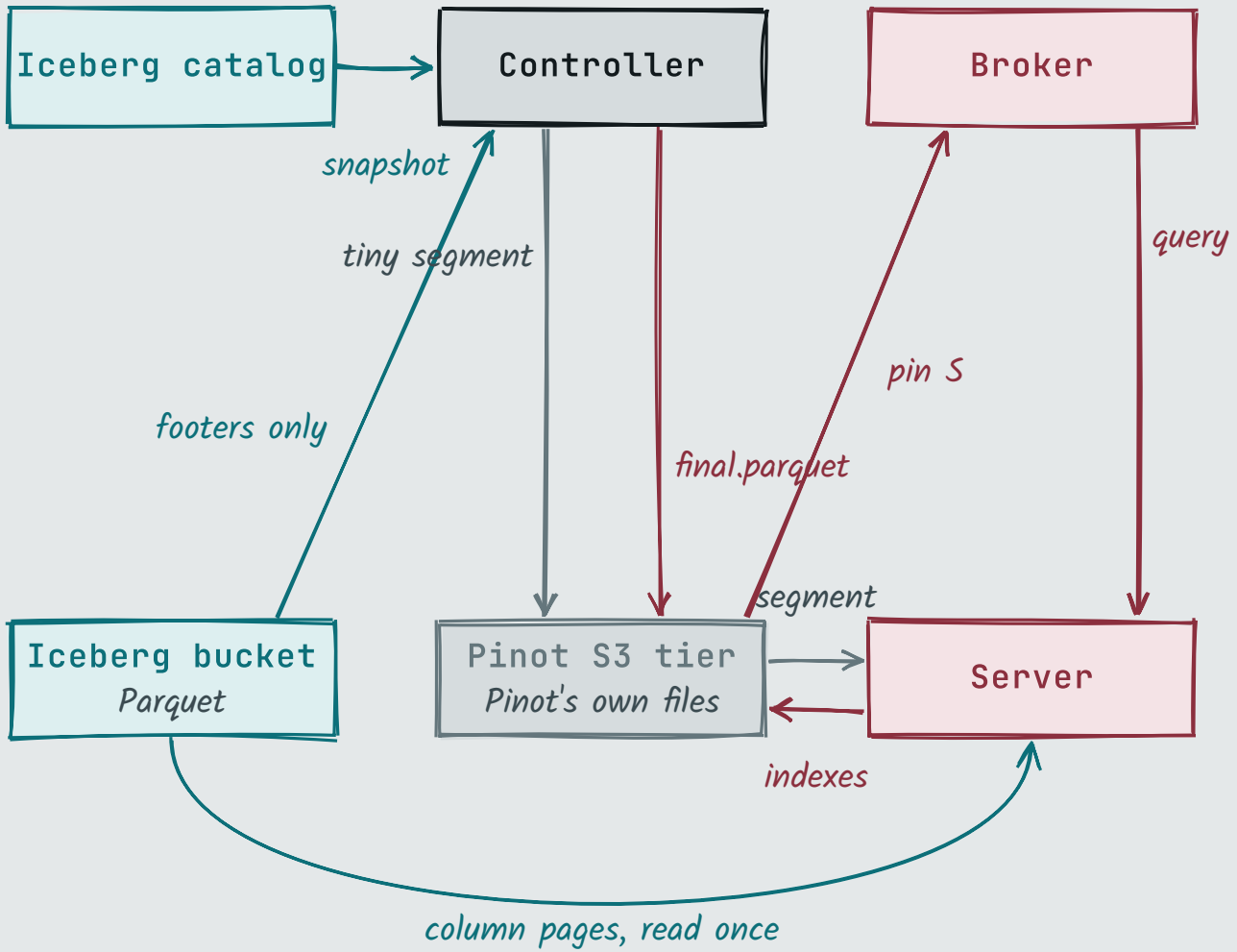
One Parquet file, one Pinot segment

- Name = hash of the file path
- Paths never change content
- Segment holds no data
- Its forward index is the Parquet file



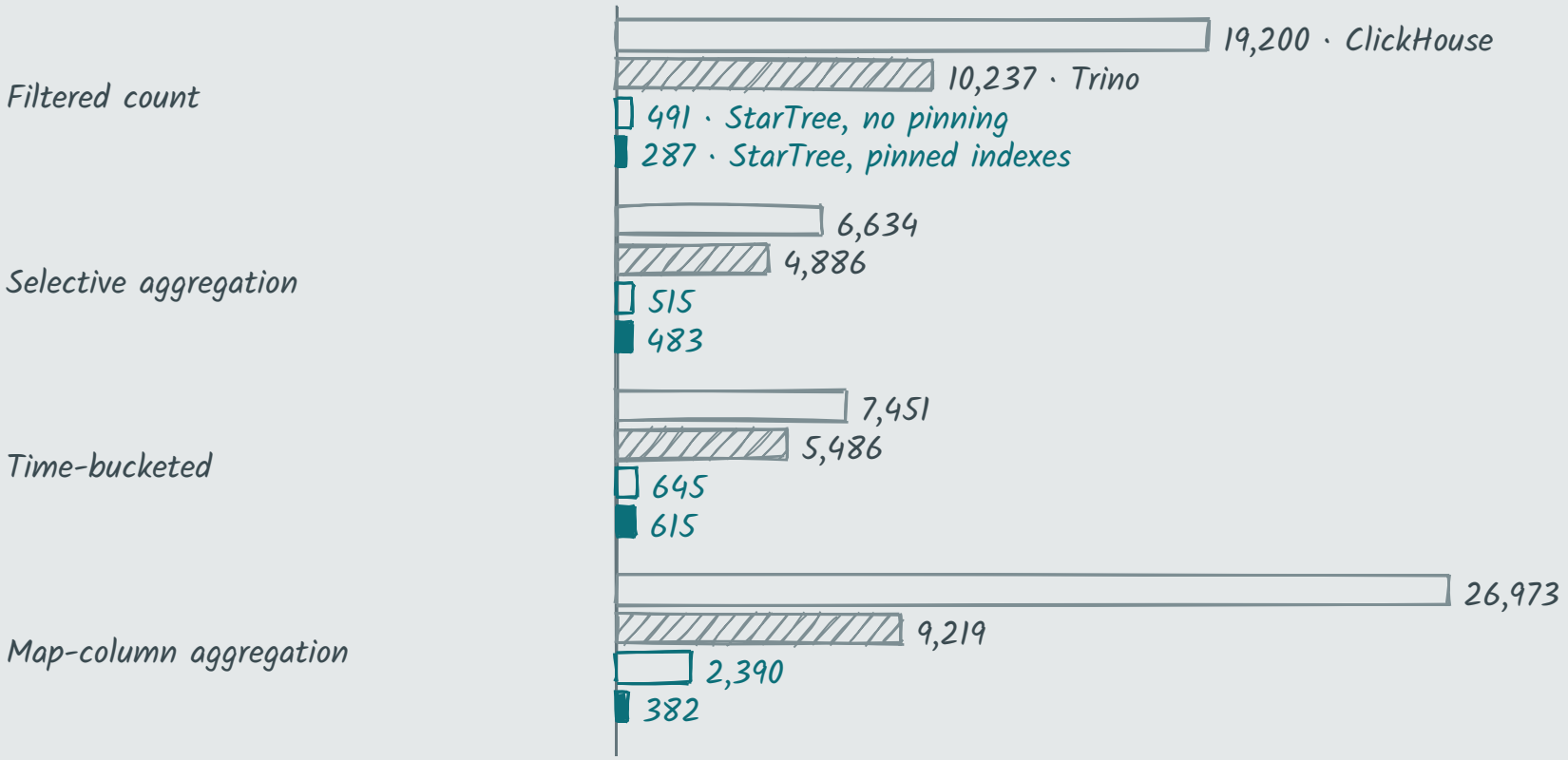
From catalog to query

1. Controller reads the snapshot
2. New file: footer only, tiny segment
3. Server builds indexes, stores them on S3
4. Snapshot published
5. Broker pins it per query



Same Parquet, same hardware

LATENCY, MS · NO DATA CACHE · LOWER IS BETTER



NETWORK IN

4.2 GB

StarTree

~20 GB

Trino · ClickHouse

StarTree benchmark: 12.2 B rows, 770 GB Parquet on S3, 4 × r6gd.4xlarge for every engine. Trino 476 (Iceberg connector), ClickHouse 26.2 (icebergS3Cluster). Indexes add 270 GB.

startree.ai · vs Trino, ClickHouse

Filtering with an index

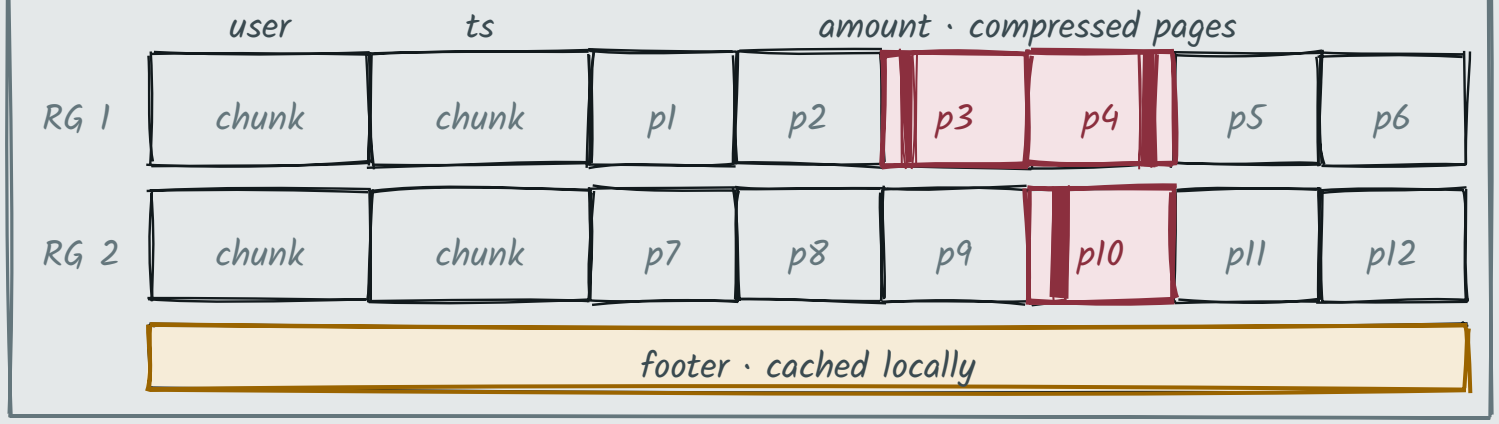
```
SELECT SUM(amount) FROM plays
WHERE user = 'bob'
```

SERVER · LOCAL DISK

footer + page index (docs → bytes)

pinned: user dictionary + inverted

S3 · data-0042.parquet · NOT OURS



BYTES FROM S3

0.5 MB

of 512 MB · **tiered segment: ~24 KB**

3 ranged GETs, whole pages

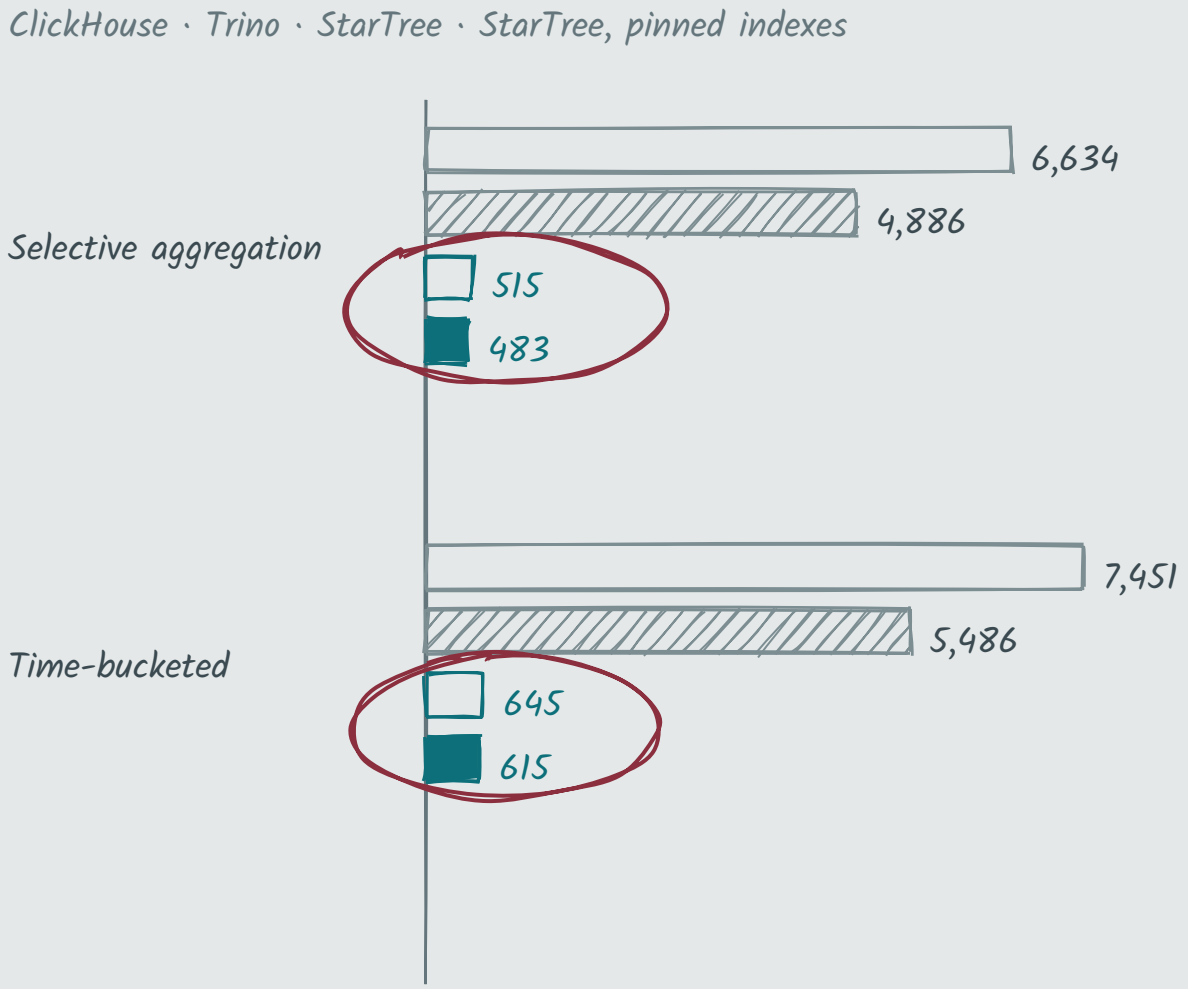
PAGE INDEX

docs 20,000–20,999 → p3, p4

docs 61,000–61,999 → p10

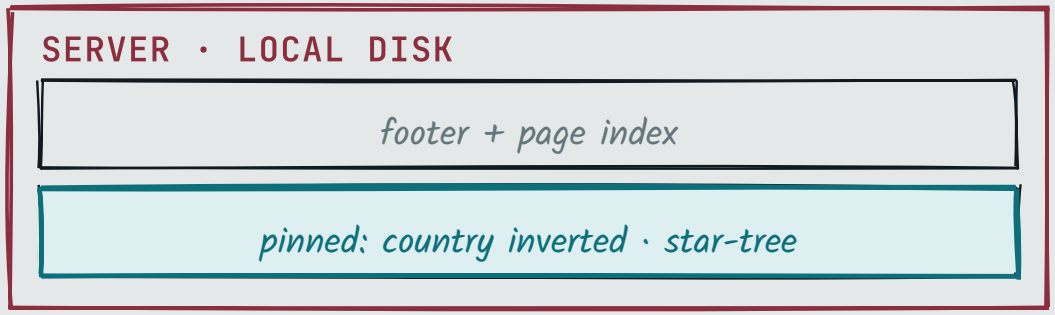
startree.ai

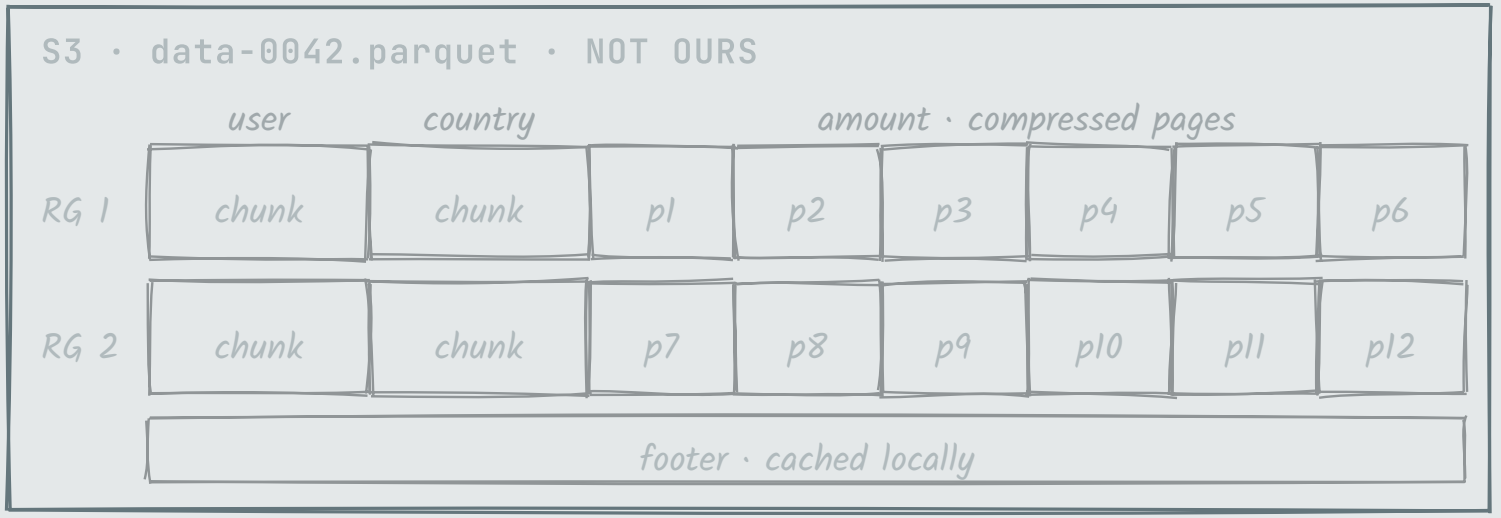
Inverted indexes don't help aggregations



Star-tree index

SELECT SUM(amount) FROM plays
WHERE country = 'ES'





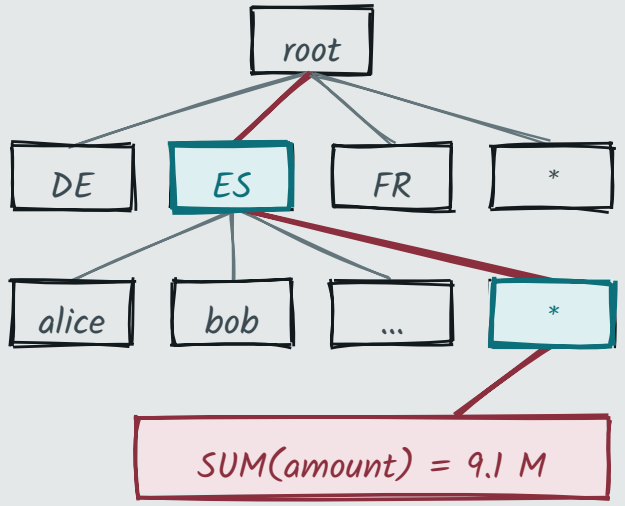
BYTES FROM S3

0

star-tree pinned locally
rows read: ***1 per segment*** *· 20 k*

Star-tree: ES → any user → one record

STAR-TREE · (country, user)



StarTree Indexes in action

- A different benchmark
- Similar query: filter + `SUM`
- Star-tree: pre-aggregated rows
- ~2,850× fewer rows read

ROWS READ PER QUERY · TPC-H SF100, 600 M ROWS

Range indexes only



~11.4 M

inferred: 114,160 measured on SF1 × 100

+ star-tree



4,002 · ~2,850× fewer

~12 k QPS

at 148 ms P90, 12 servers

Not the Trino and ClickHouse benchmark: TPC-H Q6 on Parquet in S3, External Tables, 12 × i4g.4xlarge.

startree.ai · 125k QPS on open tables

CHAPTER 4

Wrapping up

When it fits, and how to use it well

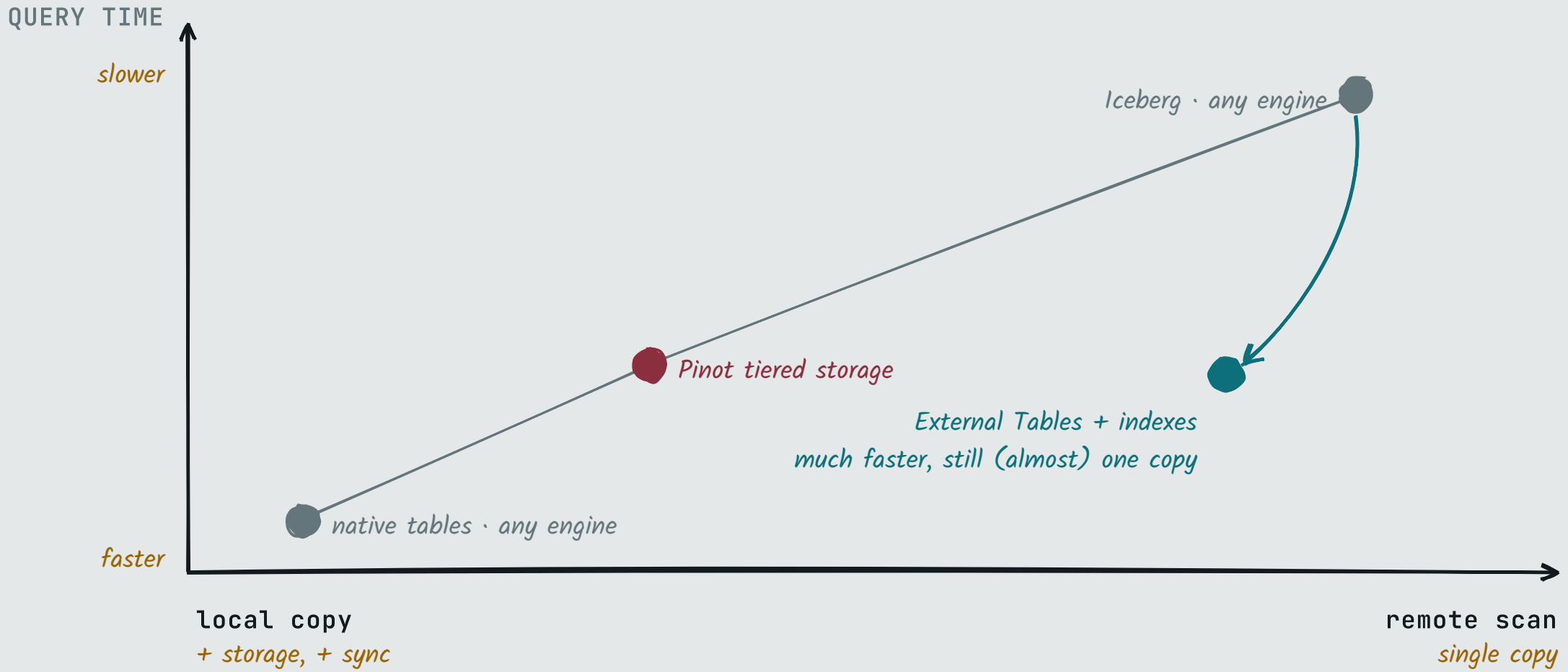


How engines read Iceberg, completed

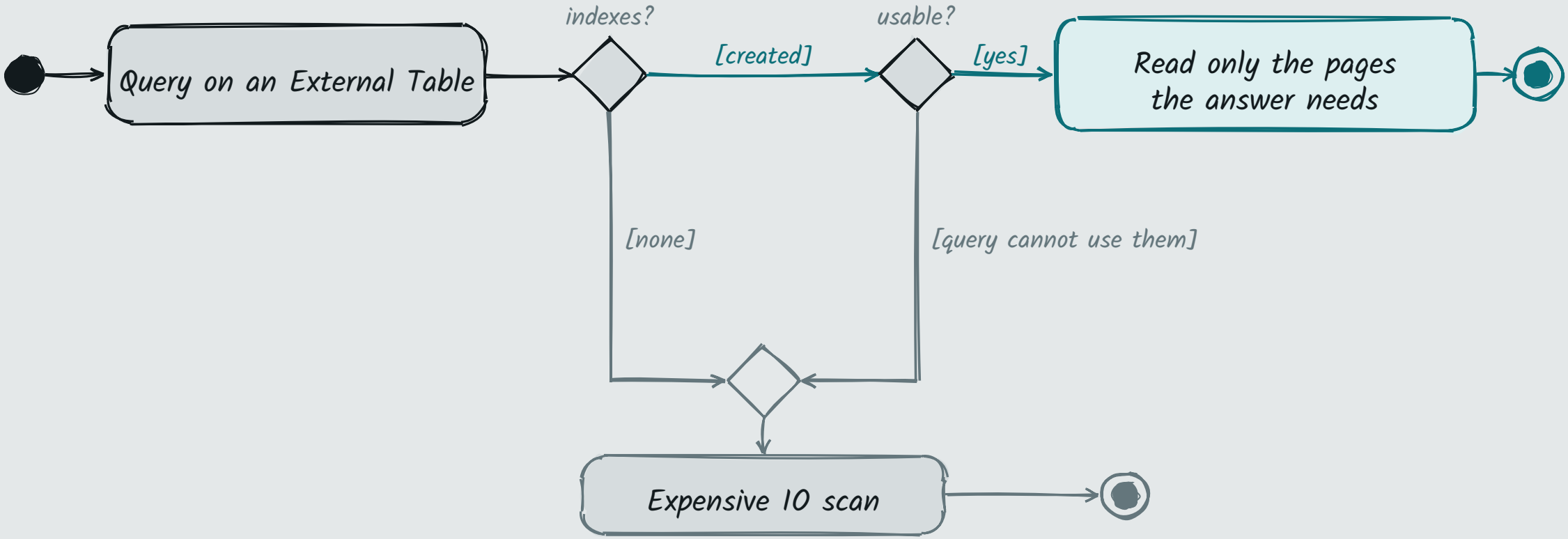
ENGINE	LOCAL COPY	OWN INDEXES ON REMOTE DATA	REMOTE SCAN + CACHE
StarTree Pinot	✓ native	✓ External Tables + indexes	✓ External Tables
Snowflake	✓	✓ search optimization*	✓
Starburst (Warp Speed)	—	✓ index layer	✓
BigQuery	✓	? external: ✗	✓
Databricks	✓	? Reyden undisclosed	✓
Trino / Athena / Spark	—	✗	✓
DuckDB / Dremio	✓	✗	✓
StarRocks / Doris / ClickHouse	✓	✗	✓

* Snowflake-managed Iceberg tables confirmed; externally managed not documented. Official docs, checked 2026-10-01.

You cannot beat physics

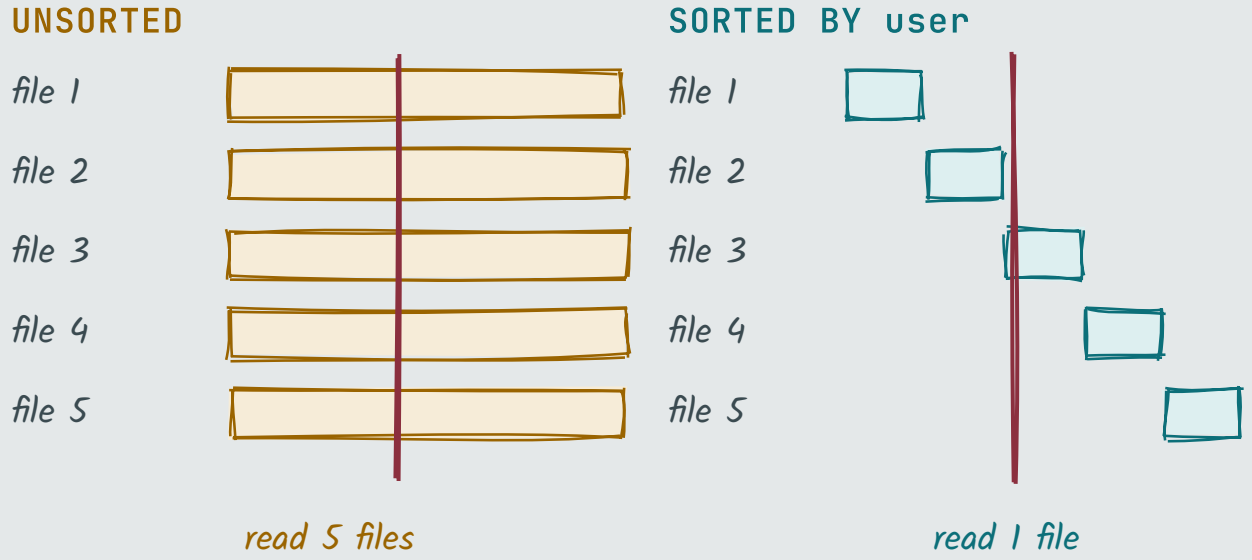


You cannot always use indexes



Write Iceberg for its readers

- Sort by the filter key
- Sane file and row-group sizes
- v3 deletion vectors
- Compact on a schedule



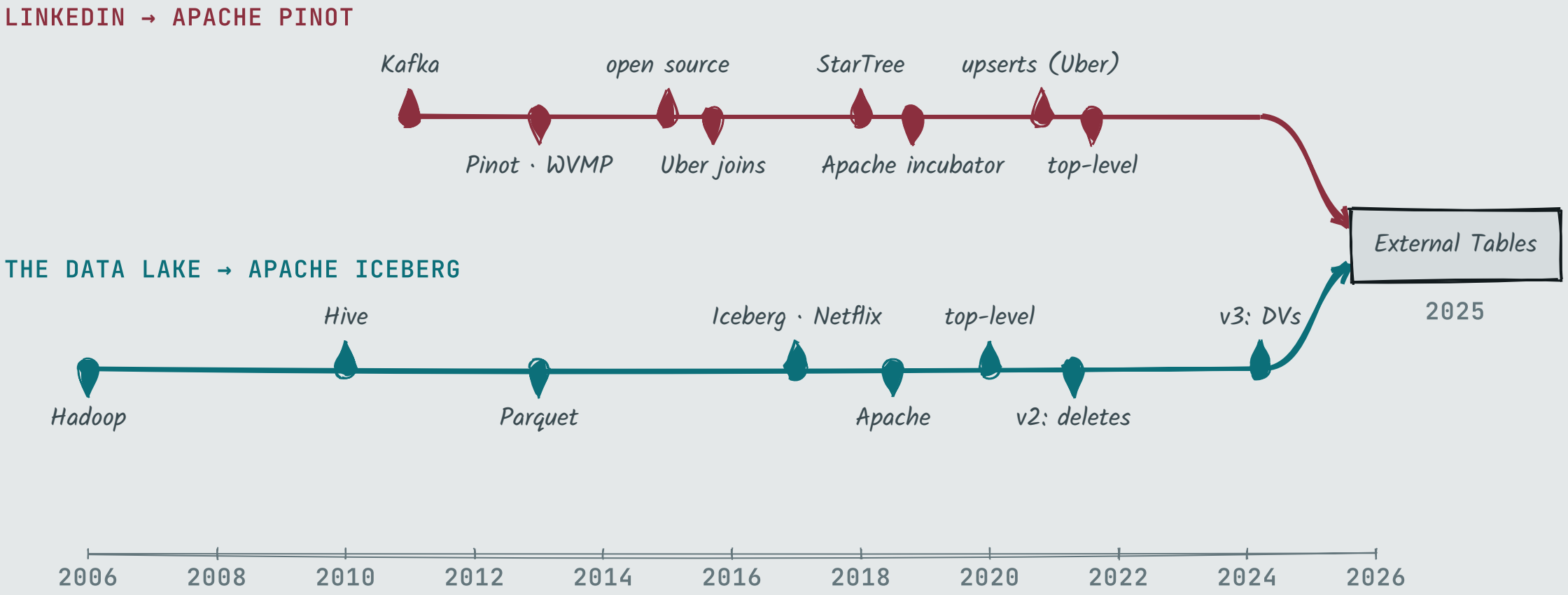
also: target file size · row-group size

prefer v3 deletion vectors

compact on a schedule

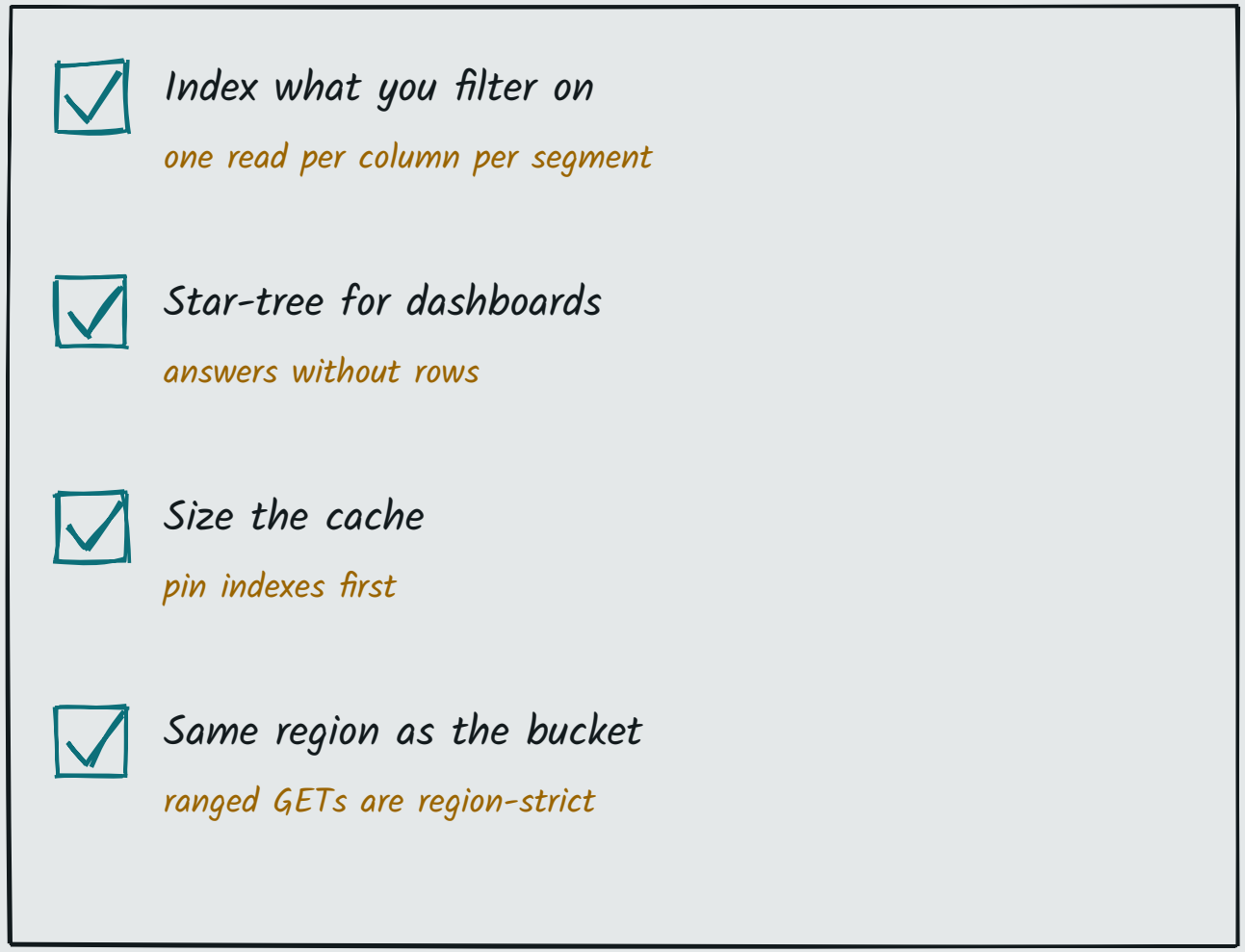
helps every engine, not only Pinot

Two timelines that meet



Configure Pinot for your queries

- Index what you filter on
- Star-tree for dashboards
- Size the cache
- Same region as the bucket



Links

gortizja.com/talks

These slides

youtube.com

Iceberg in 10 minutes

inviter.co

Apache Pinot Slack

pinot.apache.org

Apache Pinot

engineering.atspotify.com

Spotify: indexing the lake

startree.ai

Benchmark vs Trino, ClickHouse

startree.ai

125k QPS on open tables

startree.ai

StarTree Pinot on Iceberg